

# HANOI UNIVERSITY OF SCIENCE AND TECHNOLOGY

## SCHOOL OF INFORMATION AND COMMUNICATIONS TECHNOLOGY



### Solving the problem of Assingment and Scheduling with Cost and Precedence Constraints

**Authors:**      Luu Hai Dang - 202516533  
                     Duong The Hung - 202516657  
                     Le Mai Trang - 202516682  
                     Hoang Quang Bach - 202516642

**Supervisor:**   Bui Quoc Trung

**Date of Submission:** June 24, 2026

## **Abstract**

In this report, we concerntrate on addressing the problem of assigning and scheduling tasks for a multi-team project, considering team availability, task durations, execution costs, and prerequisites. The objective is to maximize the number of scheduled tasks, minimize the completion time of the project, and minimize the total cost, in the priority order. We approach many methods, each designed for a specific purpose, with varying computational costs and differing levels of efficiency under real-world constraints..

# Contents

|          |                                                                       |           |
|----------|-----------------------------------------------------------------------|-----------|
| <b>1</b> | <b>Introduction</b>                                                   | <b>4</b>  |
| <b>2</b> | <b>Problem Definition</b>                                             | <b>5</b>  |
| <b>3</b> | <b>Exact Methods</b>                                                  | <b>6</b>  |
| 3.1      | Backtracking . . . . .                                                | 6         |
| 3.1.1    | Core idea . . . . .                                                   | 6         |
| 3.1.2    | Pseudo code . . . . .                                                 | 7         |
| 3.2      | Mixed-Integer Linear Programming (MILP)/ Constraint Programming . . . | 9         |
| 3.2.1    | Decision variables . . . . .                                          | 9         |
| 3.2.2    | Objective functions . . . . .                                         | 9         |
| 3.2.3    | Constraints . . . . .                                                 | 10        |
| 3.2.4    | Important Note . . . . .                                              | 11        |
| 3.3      | Results . . . . .                                                     | 11        |
| <b>4</b> | <b>Heuristics</b>                                                     | <b>13</b> |
|          | <b>Heuristics</b>                                                     | <b>13</b> |
| 4.1      | Greedy Algorithm . . . . .                                            | 13        |
| 4.1.1    | Task Selection . . . . .                                              | 13        |
| 4.1.2    | Team Assignment . . . . .                                             | 15        |
| 4.1.3    | Experimental Parameter Tuning . . . . .                               | 16        |
| 4.1.4    | Results . . . . .                                                     | 17        |
| 4.2      | Simple/Steepest-Ascent Hill Climbing . . . . .                        | 17        |
| <b>5</b> | <b>Meta-heuristic</b>                                                 | <b>18</b> |
| 5.1      | Shared Algorithmic Components . . . . .                               | 18        |
| 5.1.1    | Schedulable Order Computation . . . . .                               | 18        |
| 5.1.2    | Solution Representation . . . . .                                     | 19        |
| 5.1.3    | Initial Solution Construction . . . . .                               | 20        |
| 5.1.4    | Decode Solution and Fitness Function . . . . .                        | 20        |
| 5.1.5    | Precedence Feasibility Check . . . . .                                | 22        |
| 5.1.6    | Neighbor Generation . . . . .                                         | 23        |
| 5.2      | Genetic Algorithm . . . . .                                           | 24        |
| 5.2.1    | Chromosome Representation . . . . .                                   | 24        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 5.2.2    | Population Initialization . . . . .                      | 24        |
| 5.2.3    | Decode Chromosome and Fitness Function . . . . .         | 25        |
| 5.2.4    | Selection Method . . . . .                               | 26        |
| 5.2.5    | Crossover Operator . . . . .                             | 27        |
| 5.2.6    | Mutation . . . . .                                       | 30        |
| 5.2.7    | Evaluate and Replace . . . . .                           | 31        |
| 5.2.8    | Termination Condition . . . . .                          | 31        |
| 5.2.9    | Main Loop . . . . .                                      | 31        |
| 5.3      | Simulated Annealing . . . . .                            | 32        |
| 5.3.1    | State Representation . . . . .                           | 32        |
| 5.3.2    | Create Initial Feasible Solution . . . . .               | 33        |
| 5.3.3    | Decode State and Fitness Function . . . . .              | 33        |
| 5.3.4    | Neighbor Generation . . . . .                            | 33        |
| 5.3.5    | Acceptance Probability . . . . .                         | 33        |
| 5.3.6    | Termination Condition . . . . .                          | 34        |
| 5.3.7    | Cooling Schedule and Main Loop . . . . .                 | 34        |
| 5.4      | Tabu Search . . . . .                                    | 35        |
| 5.4.1    | State and Move Representation . . . . .                  | 36        |
| 5.4.2    | Create Initial Feasible State . . . . .                  | 36        |
| 5.4.3    | Decode State and Fitness Calculation . . . . .           | 36        |
| 5.4.4    | Neighbor Generation . . . . .                            | 36        |
| 5.4.5    | Move Identification and Tabu Check . . . . .             | 38        |
| 5.4.6    | Termination Condition . . . . .                          | 39        |
| 5.4.7    | Main Loop . . . . .                                      | 39        |
| 5.5      | Ant Colony Optimization . . . . .                        | 40        |
| 5.5.1    | State Representation . . . . .                           | 40        |
| 5.5.2    | Create Initial Feasible Solution . . . . .               | 41        |
| 5.5.3    | Decode State and Fitness Function . . . . .              | 41        |
| 5.5.4    | Pheromone Representation and Initialization . . . . .    | 41        |
| 5.5.5    | Ant Solution Construction . . . . .                      | 41        |
| 5.5.6    | Pheromone Update . . . . .                               | 43        |
| 5.5.7    | Termination Condition . . . . .                          | 44        |
| 5.5.8    | Main Loop . . . . .                                      | 44        |
| 5.6      | Results . . . . .                                        | 45        |
| 5.6.1    | Score Comparison . . . . .                               | 46        |
| 5.6.2    | Observations . . . . .                                   | 46        |
| <b>6</b> | <b>Additional Algorithms</b>                             | <b>47</b> |
| 6.1      | Iterated Local Search . . . . .                          | 47        |
| 6.1.1    | State Representation . . . . .                           | 48        |
| 6.1.2    | Create Initial Feasible State . . . . .                  | 48        |
| 6.1.3    | Decode State and Fitness Calculation . . . . .           | 48        |
| 6.1.4    | Perturbation Mechanism . . . . .                         | 48        |
| 6.1.5    | Local Search via Variable Neighborhood Descent . . . . . | 49        |
| 6.1.6    | Acceptance Criterion . . . . .                           | 51        |

|          |                                                          |           |
|----------|----------------------------------------------------------|-----------|
| 6.1.7    | Main Loop . . . . .                                      | 52        |
| 6.2      | Variable Neighborhood Search . . . . .                   | 53        |
| 6.2.1    | State Representation . . . . .                           | 53        |
| 6.2.2    | Create Initial Feasible State . . . . .                  | 54        |
| 6.2.3    | Decode State and Fitness Calculation . . . . .           | 54        |
| 6.2.4    | Neighborhood Structures . . . . .                        | 54        |
| 6.2.5    | Shaking Procedure . . . . .                              | 55        |
| 6.2.6    | Local Search via Variable Neighborhood Descent . . . . . | 56        |
| 6.2.7    | Neighborhood Change and Acceptance Criterion . . . . .   | 57        |
| 6.2.8    | Main Loop . . . . .                                      | 58        |
| 6.3      | Results . . . . .                                        | 58        |
| 6.3.1    | Score Comparison . . . . .                               | 59        |
| 6.3.2    | Observations . . . . .                                   | 59        |
| <b>7</b> | <b>Comparison</b>                                        | <b>61</b> |
| 7.1      | Solution Quality and Optimality . . . . .                | 61        |
| 7.2      | Execution Time and Computational Complexity . . . . .    | 61        |
| 7.3      | Scalability and Generality . . . . .                     | 62        |
| 7.4      | Implementation and Tuning Effort . . . . .               | 62        |

# Chapter 1

## Introduction

The Assignment and Scheduling Problem is a fundamental combinatorial optimization problem in the fields of Operations Research and Computer Science. In its classic formulation, it addresses the optimal allocation of a set of tasks to a set of agents or resources to minimize total cost or maximize overall efficiency. However, real-world project management scenarios frequently introduce complex operational constraints—such as task dependencies, team availability time-windows, and non-overlapping schedules. These factors transform the basic model into a highly challenging, NP-hard scheduling and allocation problem.

This report focus on solving scheduling problem where a project has  $N$  tasks and  $M$  teams. Each team has an available time and cost when this team performs a particular task. In addition, there are some pairs of task which bind each other, means that a task can be able to start only if another task is completed. The purpose of this problem is generate a schedule for performing  $N$  tasks (assign a team for each task and specify the starting time-point for each that task) optimizing following criteria (prioritize in the given order):

1. Number of tasks are scheduled is maximal
2. The completion time of all tasks is minimal
3. The total cost for the task assignment is minimal

# Chapter 2

## Problem Definition

Let the project consist of  $M$  tasks, indexed by  $i = 1, 2, \dots, M$ , and  $N$  teams, indexed by  $j = 1, 2, \dots, N$ .

- $d(i)$ : duration of task  $i$
- $s(j)$ : available starting time of team  $j$
- $c(i, j)$ : cost for team  $j$  to perform task  $i$
- $Q = \{(i, j)\}$ : precedence constraints, meaning task  $j$  can start only after task  $i$  is completed

The objective is to determine:

- A mapping of each task  $i$  to a team  $j$  that can perform it
- A start time for each task

Subject to the following constraints:

1. Precedence constraints: if  $(i, j) \in Q$ , then task  $j$  must start after task  $i$  ends
2. Team availability: task  $i$  cannot be assigned to team  $j$  before  $s(j)$
3. Each team can only perform one task at a time

The scheduling must prioritize:

1. Maximizing the number of tasks scheduled
2. Minimizing the total completion time (i.e., latest finishing time)
3. Minimizing the total cost

# Chapter 3

## Exact Methods

### 3.1 Backtracking

**Backtracking** is one of the most classical algorithm to solve many problems with exact result, and is also the foundation of some complex and efficient algorithms. This algorithm gradually builds the completed results from partial ones and always find the optimal solution.

#### 3.1.1 Core idea

The core idea of Backtracking algorithm for this problem is looking for all possible results. With each result found, compare it with the best one so far. If the new result is better, consider it as the new best result. After all feasible results are checked, we can claim that the optimal solution of this problem is the value of best result. The main steps of this algorithm are:

- Initialise a "best result".
- Build the completed schedules iteratively by, respectively, selecting a pair of task and team  $(i, j)$ . Task  $i$  is chosen from an ordered list of tasks (topological order)<sup>1</sup>; while task  $j$  is chosen if it can do task  $i$ .
- Check the current completed schedule with the best result. If the current one is better, update the best result; otherwise, get rid of it and continue backtracking.
- Return the optimal schedule after all possible result are checked.

Besides, the Branch and Bound technique is also applied to prune the bad situations and decrease the running time of program.

---

<sup>1</sup>In this project, the topological order (stored in the array order) serves as a deterministic sequence for the backtracking algorithm. It guarantees that when scheduling task  $i$ , all of its prerequisite tasks have already been processed.



### 3.1.2 Pseudo code

#### Backtracking algorithm with Branch and Bound

```

1 sche <- []
2 best_sche <- []
3 best_cur <- (-1, 0, 0)
4
5 Function Backtracking(k, cn, finish_time, total_cost)
6     If k == N + 1 Then
7         cur <- (cn, -finish_time, -total_cost)
8         If cur > best_cur Then
9             best_cur <- cur
10            best_sche <- Copy(sche)
11        EndIf
12        Return
13    EndIf
14
15    best_cn, minus_best_makespan, minus_best_cost <- best_cur
16    best_makespan <- -minus_best_makespan
17    best_cost <- -minus_best_cost
18
19    remaining_tasks <- N + 1 - k
20    max_possible_cn <- cn + remaining_tasks
21
22    If max_possible_cn < best_cn Then
23        Return
24    EndIf
25
26    If max_possible_cn == best_cn Then
27        If finish_time > best_makespan Then
28            Return
29        ElseIf finish_time == best_makespan And total_cost >=
30            best_cost Then
31            Return
32        EndIf
33    EndIf
34
35    i <- order[k]
36    can_do <- True
37    latest_end_i <- 0
38
39    For h <- 1 To N Do
40        If order_tasks[h][i] == -1 Then
41            can_do <- False
42            Break
43        ElseIf order_tasks[h][i] > 0 Then
44            latest_end_i <- Max(latest_end_i, order_tasks[h][i])

```

```

44     EndIf
45 EndFor
46
47 If can_do == True Then
48     available_teams <- { h in [1, M] | cost_table[i][h] !=
49         infinity }
50     Sort available_teams ascending by (team_busy[h].last.end,
51         cost_table[i][h])
52
53     For Each j In available_teams Do
54         start <- Max(latest_end_i, team_busy[j].last.end)
55         end <- start + task_duration_time[i]
56
57         overlap <- False
58         For Each (s, e) In team_busy[j] Do
59             If Not (start >= e Or end <= s) Then
60                 overlap <- True
61                 Break
62             EndIf
63         EndFor
64
65         If overlap == False Then
66             Append(team_busy[j], (start, end))
67             Append(sche, (i, j, start))
68
69             old_deps <- []
70             For v <- 1 To N Do
71                 If order_tasks[i][v] == -1 Then
72                     order_tasks[i][v] <- end
73                     Append(old_deps, v)
74                 EndIf
75             EndFor
76
77             Backtracking(k + 1, cn + 1, Max(finish_time, end),
78                 total_cost + cost_table[i][j])
79
80             For Each v In old_deps Do
81                 order_tasks[i][v] <- -1
82             EndFor
83             Pop(sche)
84             Pop(team_busy[j])
85         EndIf
86     EndFor
87 EndIf
88
89 Backtracking(k + 1, cn, finish_time, total_cost)

```

```

88 End Function
89
90 Backtracking(1, 0, 0, 0)

```

## 3.2 Mixed-Integer Linear Programming (MILP)/ Constraint Programming

Because the number of tasks, teams, prerequisites, the duration time of each task, and the costs are all positive integer numbers, it is very perfect to apply **MILP** or **Constraint Programming** to address this problem. To find the optimal solution, the problem must be modeled by a **system of decision variables, objective functions, and constraints**. After that, all of them will be pushed into a software/tool which specializes in solving linear optimization and scheduling problems (e.g ORTools of Google). This method is able to handle a larger input and run much faster than the Backtracking algorithm. The decision variables, objective functions, and constraints of the problem are given as follows:

### 3.2.1 Decision variables

Firstly, it is necessary to create three **sets**:

- $A = \{1, 2, \dots, N\}$ : set of tasks.
- $B = \{1, 2, \dots, M\}$ : set of teams.
- $C \subset A \times A$ : set of pairs of teams (i, h) which task h can be start after the completion of task i.

Now, the **decision variables** can be defined:

- $x_{ij} \in \{0, 1\}$  for  $i \in A$  and  $j \in B$ : binary variable, equals 1 if task i is assigned to team j, and 0 otherwise.
- $y_i \in \{0, 1\}$  for  $i \in A$ : binary variable, equals 1 if task i is scheduled and 0 otherwise.
- $t_i \geq 0$  : for  $i \in A$ : the starting time of task i.
- $C_{max} \geq 0$ : the completion time of the project (makespan).
- $z_{ik} \in \{0, 1\}$  for  $(i, k) \in A \times A$ : complement variable, equals 1 if task i is performed before task k by the same team, and 0 otherwise.

### 3.2.2 Objective functions

1. **Priority 1**: Maximize the number of tasks scheduled

$$\max \quad F_1 = \sum_{i \in A} y_i$$

2. **Priority 2:** Minimize the completion time of project

$$\min \quad F_2 = C_{max}$$

3. **Priority 3:** Minimize the total cost

$$\min \quad F_3 = \sum_{i \in A} \sum_{j \in B} c_{ij} x_{ij}$$

### 3.2.3 Constraints

1. **The consistency of Assignment:** Each scheduled task must be performed by exactly one team

$$\sum_{j \in B} x_{ij} = y_i \quad \forall i \in A$$

$$x_{ij} = 0 \quad \forall (i, j) \in A \times B \text{ if } c_{ij} = \infty$$

2. **Team must be ready before doing task:** If task i is assigned to team j, it must be executed after the available time of team j

$$t_i \geq s_j \cdot x_{ij} \quad \forall (i, j) \in A \times B$$

3. **Precedence between tasks:** If task h must be done before task i, task i can only be executed after the completion time of task h

$$t_i \geq t_h + d_h - L^2(2 - y_h - y_i) \quad \forall (h, i) \in C$$

4. **Team can only perform at most one task at a time:** If task i and task k are performed by the same team j, they must not ran simultaneously:

---

<sup>2</sup>L is a large constant (Big-M). In this problem, to ensure the running time and the accurate of solution, L will be identified by sum of total duration of N tasks and the latest available time among M teams

$$t_i + d_i \leq t_k + L(1 - z_{ik}) + L(2 - x_{ij} - x_{kj}) \quad \forall i, k \in A(i \neq k), \forall j \in B$$

$$t_k + d_k \leq t_i + L \cdot z_{ik} + L(2 - x_{ij} - x_{kj}) \quad \forall i, k \in A(i \neq k), \forall j \in B$$

5. **The makespan definition:** The completion time of project is the maximum of finish time among N tasks

$$C_{max} \geq t_i + d_i - L(1 - y_i) \quad \forall i \in A$$

### 3.2.4 Important Note

Since this problem has three objectives to optimize, another new important problem is, **how to handle these objectives properly**. Here are two ways to address it:

1. **Multi-objective processing.**
  - After all decision variables and constraints are pushed into the solver, solve **Priority 1** first
  - Next, fix the optimal value found and push it into solver (consider it as a constraint) and continue solving the **Priority 2**
  - The procedure stops when the final priority is solved.
2. **One-objective processing.** Instead of respectively solving each objective, all objective can be "merged" into an unique objective:

$$F = w_1.F_1 + w_2.F_2 + w_3.F_3$$

which  $w_i$  is the weight of function  $F_i$ . The more  $F_i$  is prioritized, the bigger  $w_i$  is.

## 3.3 Results

Here is the results of Exact Method with given test cases, each result are presented as a tuple of 3 parameters: number of scheduled tasks, the finish time and the total cost.

| No. | Tasks | Teams | Prec. | Jury              | Backtracking     | LP               | CP               |
|-----|-------|-------|-------|-------------------|------------------|------------------|------------------|
| 1   | 5     | 6     | 4     | (5, 235, 145)     | (5, 235, 145)    | (5, 235, 145)    | (5, 235, 145)    |
| 2   | 10    | 15    | 6     | (10, 210, 430)    | (10, 190, 510)   | (10, 190, 480)   | (10, 190, 480)   |
| 3   | 100   | 80    | 100   | (100, 550, 4510)  | <b>NO OUTPUT</b> | (100, 550, 1850) | (100, 550, 1850) |
| 4   | 200   | 80    | 150   | (200, 370, 13240) | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> |
| 5   | 500   | 200   | 650   | (493, 840, 38150) | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> |
| 6   | 600   | 300   | 750   | (599, 550, 42300) | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> |
| 7   | 700   | 300   | 750   | (657, 510, 46270) | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> |
| 8   | 800   | 400   | 750   | (786, 530, 63960) | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> |
| 9   | 900   | 400   | 750   | (884, 820, 74300) | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> |
| 10  | 1000  | 400   | 1750  | (980, 770, 68520) | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> | <b>NO OUTPUT</b> |

# Chapter 4

## Heuristics

A heuristic is an algorithm designed to yield a potentially suboptimal or approximate solution within a significantly reduced computational time. In this chapter, we propose a heuristic-guided greedy algorithm to replace slow exact methods on large datasets.

### 4.1 Greedy Algorithm

The algorithm constructs the schedule step-by-step. At each iteration, a greedy decision is made based on two heuristic metrics:

- **Step 1 (Task Selection):** A graph-based heuristic function evaluates and selects the most critical or urgent task.
- **Step 2 (Team Assignment):** The algorithm greedily assigns the task to the best team based on two criteria: earliest start time and minimum cost. This decision is final with no backtracking.

#### 4.1.1 Task Selection

##### Overview

Each candidate task  $i$  is evaluated based on two primary criteria:

- **Successor Dependency:** The number of dependent (successor) tasks that will be blocked if the current task is not completed.
- **Processing Time:** The duration required to execute the task.

A heuristic scoring function  $H_{\text{task}}(i)$  is defined based on these two criteria. The task with the highest score is prioritized for immediate execution. Essentially, this mechanism favors tasks with a large number of successor tasks and the shortest processing time, formulated as a maximization problem.

$$H_{\text{task}}(i) = w_1 \cdot |\text{Succ}(i)| + w_2 \cdot \frac{1}{d(i)}$$

$\text{Succ}(i)$ : Number of blocked successor tasks if task  $i$  is incomplete.

$d(i)$ : Processing time of task  $i$ .

## Compute Descendant Tasks

To avoid redundant graph traversals, the total number of descendant tasks is precomputed using a memoized Depth-First Search (DFS) combined with bitset arrays. During execution, this reduces the operation to a simple bit-counting procedure.

The function below returns the decimal representation of the bitset string for each individual node. A bit is set to 1 if the current node can reach or visit that specific node (following the task sequence). Otherwise, the bit is set to 0.

Bitset string for each individual node

```

1 FUNCTION get_reachable_set(node, adj, memo, visited)
2   if visited[node] then
3     return memo[node]
4   and if
5     visited[node] <- true
6     res_set <- 0
7     for each neighbor in adj[node] do
8       neighbor_set <- get_reachable_set(neighbor, adj, memo,
9         visited)
10      res_set <- res_set or neighbor_set
11      res_set <- res_set or (1 shilft left neighbor)
12    end for
13    memo[node] <- res_set
14    return res_set
15 end FUNCTION

```

The function below returns the number of 1-bits (in set bits). This total directly represents the number of successor tasks (descendants) linked to the current node.

Task successors count

```

1 FUNCTION compute_successors_dp(N, adj)
2   memo <- array of size (N + 1) initialized to 0
3   visited <- array of size (N + 1) initialized to false
4   succ_count <- array of size (N + 1) initialized to 0
5   for i <- 1 to N do
6     if not visited[i] then
7       get_reachable_set(i, adj, memo, visited)
8     end if
9   end for
10  for i <- 1 to N do
11    succ_count[i] <- count_set_bits(memo[i])
12  end for
13  return succ_count
14 end FUNCTION

```

## Pseudo Code



```

1 FUNCTION select_tasks_priority_heu1(ready_tasks, succ_count,
  durations, task_teams, w1 <- 0.3, w2 <- 0.7)
2   task_scores <- empty list
3   for each task in ready_tasks do
4     if not task_teams[task] then
5       continue
6     end if
7     if durations[task] > 0 then
8       d_i <- durations[task]
9     else
10      d_i <- 1e-9
11    end if
12    score <- w1 * succ_count[task] + w2 * (1.0 / d_i)
13    append (score, task) to task_scores
14  end for
15  sort task_scores by score descending, task ascending
16  sorted_tasks <- empty list
17  for each (score, task) in task_scores do
18    append task to sorted_tasks
19  end for
20  return sorted_tasks
21 end FUNCTION

```

**Outcome:** The execution priority order of the tasks is established. The subsequent step involves selecting the appropriate team for execution -> Heuristic 2.

### 4.1.2 Team Assignment

#### Overview

For a given task  $i$ , we evaluate the fitness of each candidate team  $j$  based on two criteria:

- **Start Time:** The earliest time team  $j$  can start working on task  $i$ .
- **Cost:** The total cost if task  $i$  is assigned to team  $j$ .

A scoring function  $H_{team}(i, j)$  is used to calculate a score for each team. The team with the lowest score will be chosen and assigned to the task previously selected by Heuristic 1. This mechanism prioritizes teams that can start early (to avoid blocking the next tasks) and have a low cost. Therefore, this step operates as a minimization problem.

$$H_{team}(i, j) = w_3 \cdot \text{actual\_start\_time} + w_4 \cdot c(i, j)$$

$\text{actual\_start\_time}(i, j)$ : The earliest time team  $j$  can begin executing task  $i$ .

$c(i, j)$ : The cost incurred when assigning task  $i$  to team  $j$ .

## Pseudo Code

```

1 FUNCTION assign_team_heu2(task, rev_adj, comp_time, task_teams,
  team_busy, s, w3 <- 0.8, w4 <- 0.2)
2   if rev_adj[task] is empty then
3     task_ready_time <- 0
4   else
5     task_ready_time <- max(comp_time[p] for each p in rev_adj[
      task])
6   end if
7   options <- empty list
8   for each (team_id, cost) in task_teams[task] do
9     team_avail <- max(s[team_id], team_busy[team_id])
10    actual_start <- max(task_ready_time, team_avail)
11    score <- w3 * actual_start + w4 * cost
12    append (score, team_id, actual_start) to options
13  end for
14  if options is empty then
15    return null, null
16  end if
17  (best_score, best_team, best_start) <- minimum of options chosen
      by score ascennding, team_id ascending
18  return best_team, best_start
19 end FUNCTION

```

### 4.1.3 Experimental Parameter Tuning

To ensure stable performance of the scoring functions, the weight pairs are constrained by a convex combination principle where their sum must equal 1:

$$w_1 + w_2 = 1 \text{ and } w_3 + w_4 = 1$$

#### Task Selection Weights

The number of successor tasks is much larger than the reciprocal of the duration:

$$Succ(i) \gg \frac{1}{d(i)}$$

Without the weights  $w_1$  and  $w_2$ , the scoring function would be simplifies to

$$H_{\text{task}}(i) = Succ(i).$$

Finding the values for  $w_1$  and  $w_2$  through experimental testing: The weights are varied within the range  $[0.1, 1.0]$  with a step size of 0.1.

As a results, the parameter set  $w_1 = 0.3$  and  $w_2 = 0.7$  is selected as the most optimal for the problem.

### Team Assignment Weights

Based on the problem constraints where time prioritizes over cost . We use the same experimental testing to find the most optimal value for  $w_3$  and  $w_4$  that  $w_3 = 0.8$  and  $w_4 = 0.2$ .

#### 4.1.4 Results

Here is the results of Greedy Algorithm with given testcases, each result are prenseted as a tuple of 3 parameters: number of tasks, the finish time and the total cost.

| No. | Tasks | Teams | Prec. | Jury              | Greedy            |
|-----|-------|-------|-------|-------------------|-------------------|
| 1   | 5     | 6     | 4     | (5, 235, 145)     | (5, 235, 145)     |
| 2   | 10    | 15    | 6     | (10, 210, 430)    | (10, 210, 430)    |
| 3   | 100   | 80    | 100   | (100, 550, 4510)  | (100, 550, 4510)  |
| 4   | 200   | 80    | 150   | (200, 370, 13240) | (200, 370, 13240) |
| 5   | 500   | 200   | 650   | (493, 840, 38150) | (493, 840, 38150) |
| 6   | 600   | 300   | 750   | (599, 550, 42300) | (599, 550, 42300) |
| 7   | 700   | 300   | 750   | (657, 510, 46270) | (657, 510, 46270) |
| 8   | 800   | 400   | 750   | (786, 530, 63960) | (786, 530, 63960) |
| 9   | 900   | 400   | 750   | (884, 820, 74300) | (884, 820, 74300) |
| 10  | 1000  | 400   | 1750  | (980, 770, 68520) | (980, 770, 68520) |

## 4.2 Simple/Steepest-Ascent Hill Climbing

Detailed analyses of Simulated Annealing and Tabu Search - both developed as enhancements to the Hill Climbing algorithm - are reserved for the Metaheuristic chapter.

# Chapter 5

## Meta-heuristic

Meta-heuristic methods are high-level, problem-independent frameworks that guide underlying search procedures to explore large solution spaces effectively, trading guaranteed optimality for practical scalability. This chapter presents four meta-heuristic algorithms applied to the multi-team task scheduling problem:

- **Genetic Algorithm (GA)**
- **Simulated Annealing (SA)**
- **Tabu Search (TS)**
- **Ant Colony Optimization (ACO)**

Despite their different search paradigms, all four methods share a common algorithmic backbone: a unified solution representation, the same greedy initialization, an identical decode-and-evaluate pipeline, and the same basic perturbation operators. These shared components are defined once in Section 5.1 and reused across the subsequent method-specific sections.

### 5.1 Shared Algorithmic Components

The following building blocks appear identically (or near-identically) in all four meta-heuristic implementations and are therefore defined here to avoid repetition.

#### 5.1.1 Schedulable Order Computation

Before any search begins, a fixed topological ordering of *schedulable* tasks is computed. A task is schedulable if at least one team is capable of performing it (`can_do[i] = 1`). Kahn's BFS-based topological sort is applied; only schedulable tasks are retained in the final ordering `schedulable_order`, which is then kept fixed throughout the entire search.

**Algorithm 3** Compute Schedulable Order

---

```

1: function COMPUTE_SCHEDULABLE_ORDER(num_task, precedence_graph,
   can_do)
2:   in_deg  $\leftarrow [0] \times \text{num\_task}$ 
3:   for  $u \leftarrow 0$  to  $\text{num\_task} - 1$  do
4:     for  $v \in \text{precedence\_graph}[u]$  do
5:       in_deg[v] += 1
6:     end for
7:   end for
8:   queue  $\leftarrow \text{deque}(\{i \mid \text{in\_deg}[i] = 0\})$ 
9:   full_order  $\leftarrow []$ 
10:  while queue is not empty do
11:     $u \leftarrow \text{queue.popleft}()$ 
12:    full_order.append( $u$ )
13:    for  $v \in \text{precedence\_graph}[u]$  do
14:      in_deg[v] -= 1
15:      if in_deg[v] = 0 then
16:        queue.append( $v$ )
17:      end if
18:    end for
19:  end while
20:  return [ $u \mid u \in \text{full\_order}, \text{can\_do}[u] = 1$ ]
21: end function

```

---

**5.1.2 Solution Representation**

Each candidate solution is encoded as a pair of equal-length lists:

- **task\_order**: a topologically valid permutation of schedulable tasks, specifying the order in which tasks are processed.
- **team\_assignment**: the  $k$ -th entry is the team assigned to the  $k$ -th task in **task\_order**.

**Structure:**

State / Chromosome / Solution:

```

{
  "task_order"      : [3, 1, 4, 2, 5],
  "team_assignment" : [2, 1, 3, 1, 2]
}

```

**Example:** The above encodes that tasks are executed in order  $3 \rightarrow 1 \rightarrow 4 \rightarrow 2 \rightarrow 5$ ; task 3 is done by team 2, task 1 by team 1, task 4 by team 3, task 2 by team 1, and task 5 by team 2.

### 5.1.3 Initial Solution Construction

All four methods initialise their search from a *greedy* solution. The greedy constructor processes tasks in the fixed schedulable order and, for each task, selects the team whose earliest available start time is smallest (ties broken by cost).

---

**Algorithm 4** Greedy Construction

---

```

1: function GREEDY_CONSTRUCT(schedulable_order, rev_graph, team_avail,
   duration, cost)
2:   task_End  $\leftarrow [0] \times \text{num\_task}$ 
3:   teamAvail  $\leftarrow \text{team\_avail}[:, ]$ 
4:   task_order  $\leftarrow []$ ; team_assignment  $\leftarrow []$ 
5:   for currentTask  $\in$  schedulable_order do
6:     pred_finish  $\leftarrow 0$ 
7:     for  $p \in \text{rev\_graph}[\text{currentTask}]$  do
8:       if task_End[p] > pred_finish then
9:         pred_finish  $\leftarrow \text{task\_End}[p]$ 
10:      end if
11:    end for
12:    best_choice  $\leftarrow \text{None}$ ; best_time  $\leftarrow \infty$ ; best_cost  $\leftarrow \infty$ 
13:    for (team, c)  $\in$  cost[currentTask] do
14:      earliest  $\leftarrow \max(\text{teamAvail}[\text{team}], \text{pred\_finish})$ 
15:      if earliest < best_time or (earliest = best_time and c < best_cost) then
16:        best_time  $\leftarrow$  earliest; best_cost  $\leftarrow$  c; best_choice  $\leftarrow$  team
17:      end if
18:    end for
19:    task_order.append(currentTask); team_assignment.append(best_choice)
20:    task_End[currentTask]  $\leftarrow$  best_time + duration[currentTask]
21:    teamAvail[best_choice]  $\leftarrow$  task_End[currentTask]
22:  end for
23:  return State(task_order, team_assignment)
24: end function

```

---

### 5.1.4 Decode Solution and Fitness Function

Given a solution, the decoder simulates task execution according to **task\_order**. For each task, the actual start time is determined by the earliest feasible time after considering the assigned team's availability, the completion time of all predecessors, and the finish time of the previous task assigned to the same team.

Since the problem is multi-objective, *lexicographic* fitness triple is used across all four methods:

$$\text{fitness} = \left( \underbrace{M - \text{scheduled}}_{\text{minimize}}, \underbrace{\text{completion}}_{\text{minimize}}, \underbrace{\text{total cost}}_{\text{minimize}} \right)$$

The fitness function evaluates the quality of each solution by calculating three main criteria:

- **Number of unscheduled tasks:** The number of tasks that cannot be scheduled. This criterion aims to minimize the number of unscheduled tasks, or equivalently maximize the total number of scheduled tasks.
- **Completion time:** The overall completion time of the whole schedule. This criterion aims to minimize the total execution time, also known as the makespan.
- **Assigning cost:** The overall assignment cost of all scheduled tasks. This criterion aims to minimize the total cost.

Tuples are compared lexicographically: fewer unscheduled tasks are preferred first, then smaller completion time, and finally lower total cost. The decoding procedure is described as follows:

1. Simulate task execution in `task_order`.
2. For each task:
  - Find the **earliest feasible start time** after:
    - the assigned team's availability  $s(j)$ ;
    - the completion time of all predecessor tasks;
    - the finish time of the previous task assigned to the same team.
  - If feasible, schedule the task and compute:
    - start time;
    - end time;
    - assignment cost  $c(i, j)$ .
3. Finally, keep track of:
  - total number of scheduled tasks;
  - completion time;
  - total cost.

---

**Algorithm 5** Decode Solution

---

```

1: function DECODE_SOLUTION(sol, duration, num_task, team_avail, rev_graph)
2:   task_End  $\leftarrow$  [0]  $\times$  num_task;   teamAvail  $\leftarrow$  team_avail[:]
3:   schedule  $\leftarrow$  []
4:   for (currentTask, team)  $\in$  zip(sol.task_order, sol.team_assignment) do
5:     pred_finish  $\leftarrow$  0
6:     for  $p \in$  rev_graph[currentTask] do

```

```

7:         if task_End[p] > pred_finish then
8:             pred_finish ← task_End[p]
9:         end if
10:    end for
11:    earliest ← max(teamAvail[team], pred_finish)
12:    teamAvail[team] ← earliest + duration[currentTask]
13:    task_End[currentTask] ← earliest + duration[currentTask]
14:    schedule.append((currentTask, team, earliest))
15: end for
16: return schedule
17: end function

```

---

**Algorithm 6** Calculate Fitness

---

```

1: function CALC_FITNESS(sol, num_task, cost, duration, team_avail, rev_graph)
2:   schedule ← DECODE_SOLUTION(sol, duration, num_task, team_avail,
   rev_graph)
3:   finish_task ← 0; completion ← 0; totalCost ← 0
4:   for (task, team, start) ∈ schedule do
5:     finish_task += 1
6:     finish ← start + duration[task]
7:     if finish > completion then
8:       completion ← finish
9:     end if
10:    totalCost += cost[task][team]
11:   end for
12:   return (num_task – finish_task, completion, totalCost)
13: end function

```

---

**5.1.5 Precedence Feasibility Check**

Before any permutation of tasks is accepted as a valid ordering, it must satisfy all precedence constraints.

**Algorithm 7** Precedence Feasibility Check

---

```

1: function IS_PRECEDENCE_FEASIBLE(task_order, precedence)
2:   pos ← {task : idx | (idx, task) ∈ enumerate(task_order)}
3:   for (i, j) ∈ precedence do
4:     if i ∈ pos and j ∈ pos and pos[i] ≥ pos[j] then
5:       return False
6:     end if
7:   end for
8:   return True
9: end function

```

---



### 5.1.6 Neighbor Generation

All search-based methods perturb a current solution via two complementary operators applied in sequence.

#### Task Order Mutation

Two positions  $i, j$  are randomly chosen and swapped. The swap is accepted only if the resulting order remains precedence-feasible; otherwise the attempt is retried up to **attempts** times. The corresponding team entries are swapped in tandem.

---

#### Algorithm 8 Task Order Mutation

---

```

1: function TASK_ORDER_MUTATION(task_order, team_assignment, precedence,
   attempts)
2:   if |task_order| < 2 then
3:     return task_order[:]
4:   end if
5:   for  $k \leftarrow 1$  to attempts do
6:     new_order  $\leftarrow$  task_order[:]
7:      $i, j \leftarrow$  random sample of 2 distinct indices from new_order
8:     swap(new_order[ $i$ ], new_order[ $j$ ])
9:     if IS_PRECEDENCE_FEASIBLE(new_order, precedence) then
10:      swap(team_assignment[ $i$ ], team_assignment[ $j$ ])
11:      return new_order
12:    end if
13:  end for
14:  return task_order[:]
15: end function

```

---

#### Team Assignment Mutation

Each task is independently re-assigned to a *different* capable team with probability **mutation\_rate**.

---

#### Algorithm 9 Team Assignment Mutation

---

```

1: function TEAM_ASSIGNMENT_MUTATION(task_order, team_assignment,
   available_team, mutation_rate)
2:   new_assignment  $\leftarrow$  team_assignment[:]
3:   for (idx, task)  $\in$  enumerate(task_order) do
4:     if random() < mutation_rate then
5:       viable  $\leftarrow$  available_team[task]
6:       if |viable| > 1 then
7:         old_team  $\leftarrow$  new_assignment[idx]; new_team  $\leftarrow$  old_team

```

```
8:         while new_team = old_team do
9:             new_team ← random choice from viable
10:        end while
11:        new_assignment[idx] ← new_team
12:    end if
13: end if
14: end for
15: return new_assignment
16: end function
```

---

## 5.2 Genetic Algorithm

**Introduction:** Genetic Algorithms (GA) are evolutionary algorithms inspired by natural selection. They maintain a *population* of candidate solutions and improve solution quality across generations by selecting fit individuals, combining their traits via crossover, and introducing random mutations.

**Main Idea:** By repeatedly selecting the fittest individuals, mixing their information through crossover, and mutating offspring to maintain diversity, GA balances broad exploration of the search space with focused exploitation of promising regions.

1. **Initialization:** Start with a population of candidate solutions (chromosomes).
2. **Selection:** Choose the best candidates based on a fitness function.
3. **Crossover:** Combine parts of two parents to create new offspring.
4. **Mutation:** Randomly alter some genes to maintain diversity.
5. **Replacement:** Form a new generation and repeat the process.

### 5.2.1 Chromosome Representation

GA uses the solution representation defined in Section 5.1.2: each chromosome consists of a `task_order` list together with its corresponding `team_assignment` list.

### 5.2.2 Population Initialization

The population is seeded with one greedy individual (Section 5.1.3) and  $(\text{population\_size}-1)$  randomly constructed feasible individuals. A random individual is built via a randomised topological sort over the schedulable tasks.

---

**Algorithm 10** Random Individual Construction

---

```

1: function CONSTRUCT_RANDOM(schedulable_order, precedence_graph,
   available_team)
2:   task_set  $\leftarrow$  set(schedulable_order)
3:   Compute in_deg[u] for all  $u \in$  task_set via precedence_graph
4:   ready  $\leftarrow [t \mid t \in$  schedulable_order, in_deg[t] = 0]
5:   task_order  $\leftarrow []$ ;   visited  $\leftarrow \emptyset$ 
6:   while ready  $\neq \emptyset$  do
7:     pick  $\leftarrow$  random choice from ready
8:     ready.remove(pick)
9:     if pick  $\in$  visited then
10:      continue
11:     end if
12:     visited.add(pick);   task_order.append(pick)
13:     for  $v \in$  precedence_graph[pick] do
14:       if  $v \in$  task_set then
15:         in_deg[v]  $\mathrel{--} 1$ 
16:         if in_deg[v] = 0 then
17:           ready.append(v)
18:         end if
19:       end if
20:     end for
21:   end while
22:   team_assignment  $\leftarrow$  [random choice(available_team[i])  $\mid i \in$  task_order]
23:   return Chromosome(task_order, team_assignment)
24: end function

```

---

**Algorithm 11** Population Initialization

---

```

1: function INITIALIZE_POPULATION(population_size, schedulable_order, ...)
2:   population  $\leftarrow []$ 
3:   population.append(GREEDY_CONSTRUCT(...))
4:   for  $i \leftarrow 1$  to population_size - 1 do
5:     population.append(CONSTRUCT_RANDOM(schedulable_order,
   precedence_graph, available_team))
6:   end for
7:   return population
8: end function

```

---

**5.2.3 Decode Chromosome and Fitness Function**

The chromosome is decoded and evaluated using `DECODE_SOLUTION` and `CALC_FITNESS` defined in Section 5.1.4.

### 5.2.4 Selection Method

**Rank Selection** is used to choose parent chromosomes for reproduction. Instead of using raw fitness values, the population is first sorted by fitness. Then each individual receives a probability based on its rank.

Let  $n$  be the population size and  $i$  be the rank index, where  $i = 0$  is the best individual. The probability is assigned by a linear decay:

$$p_i = \frac{1}{n} \left( sp - \frac{2(sp-1)i}{n-1} \right)$$

In this project, we choose  $sp = 1.5$ . Therefore, the best individual receives probability  $\frac{1.5}{n}$ , while the worst individual receives probability  $\frac{0.5}{n}$ . This creates a linear decay of probabilities while still keeping:

$$\sum_{i=0}^{n-1} p_i = 1$$

Parent selection is then performed using roulette-wheel selection:

1. **Sorting:** Sort the population in ascending fitness order.
2. **Probability Assignment:** Assign each individual a linearly decayed probability based on its rank.
3. **Roulette-wheel Selection:** Randomly sample parents according to these probabilities, so better individuals are more likely to be selected but weaker individuals still have a chance.

---

#### Algorithm 12 Rank Selection

---

```

1: function RANK_SELECTION(population, ...)
2:   Sort population ascending by CALC_FITNESS(·)
3:    $k \leftarrow |\text{population}|$ ;    $sp \leftarrow 1.5$ ;    $\text{probs} \leftarrow [0] \times k$ 
4:   for  $i \leftarrow 0$  to  $k - 1$  do
5:      $\text{probs}[i] \leftarrow \frac{1}{k} \left( sp - \frac{(2\,sp - 2)\,i}{k - 1} \right)$ 
6:   end for
7:   for  $i \leftarrow 1$  to  $k - 1$  do
8:      $\text{probs}[i] += \text{probs}[i - 1]$ 
9:   end for
10:   $\text{parents} \leftarrow []$ 
11:  for  $\_ \leftarrow 1$  to 2 do
12:     $r \leftarrow \text{random}()$ ;    $\text{chosen} \leftarrow \text{population}[-1]$ 

```

```
13:     for  $i \leftarrow 0$  to  $k - 1$  do
14:         if  $\text{probs}[i] \geq r$  then
15:              $\text{chosen} \leftarrow \text{population}[i]$ ; break
16:         end if
17:     end for
18:      $\text{parents.append}(\text{chosen})$ 
19: end for
20: return  $\text{parents}[0]$ ,  $\text{parents}[1]$ 
21: end function
```

---

### 5.2.5 Crossover Operator

The crossover operator is a key **genetic algorithm** mechanism that combines parts of two parent solutions to create new offspring. Inspired by biological reproduction, crossover aims to exploit good traits from both parents and pass them to the next generation.

- Take two parent chromosomes
- Mix their information to produce two more children
- Children inherit the strengths of both parents, increasing the chance of better solutions

We care about 2 main crossover:

1. Precedence-preserving Order-based Crossover (POX)
2. Crossover for Team Assignment

#### Precedence-preserving Order-based Crossover

This crossover creates a new task order while trying to preserve the relative structure of both parents and maintain precedence feasibility.

**Steps:**

1. Initialize an empty child:
2. Traverse all positions in **parent1**. With probability 0.5, copy the task in that position into the same position as **the child**.
3. Traverse **parent2** from left to right and use its tasks to fill the remaining empty positions of **the child**, skipping duplicated tasks.
4. Check if the resulting child violates precedence constraints by Section 5.1.5, repair it using topological sorting.

**Algorithm 13** Topological Repair

---

```

1: function TOPOLOGICAL_REPAIR(task_order, precedence_graph)
2:   task_set  $\leftarrow$  set(task_order)
3:   Compute in_deg[u] for all  $u \in$  task_set via precedence_graph
4:   queue  $\leftarrow$  deque( $\{t \mid t \in$  task_order, in_deg[t] = 0 $\}$ )
5:   result  $\leftarrow$  []; visited  $\leftarrow$   $\emptyset$ 
6:   while queue is not empty do
7:      $u \leftarrow$  queue.popleft()
8:     if  $u \in$  visited then
9:       continue
10:    end if
11:    visited.add( $u$ ); result.append( $u$ )
12:    for  $v \in$  precedence_graph[ $u$ ] do
13:      if  $v \in$  task_set then
14:        in_deg[ $v$ ]  $\mathrel{--}$  1
15:        if in_deg[ $v$ ] = 0 then
16:          queue.append( $v$ )
17:        end if
18:      end if
19:    end for
20:  end while
21:  return result
22: end function

```

---

**Algorithm 14** Precedence-preserving Order-based Crossover (POX)

---

```

1: function POX(parent1, parent2, precedence, precedence_graph)
2:   child  $\leftarrow$   $[-1] \times |\text{parent1}|$ ; added  $\leftarrow$   $\emptyset$ 
3:   for  $i \leftarrow 0$  to  $|\text{parent1}| - 1$  do
4:     if random()  $\geq 0.5$  then
5:       child[ $i$ ]  $\leftarrow$  parent1[ $i$ ]; added.add(parent1[ $i$ ])
6:     end if
7:   end for
8:    $j \leftarrow 0$ 
9:   for  $i \leftarrow 0$  to  $|\text{child}| - 1$  do
10:    if child[ $i$ ] = -1 then
11:      while parent2[ $j$ ]  $\in$  added do
12:         $j \mathrel{+}= 1$ 
13:      end while
14:      child[ $i$ ]  $\leftarrow$  parent2[ $j$ ]; added.add(parent2[ $j$ ]);  $j \mathrel{+}= 1$ 
15:    end if
16:  end for
17:  if  $\neg$ IS_PRECEDENCE_FEASIBLE(child, precedence) then
18:    child  $\leftarrow$  TOPOLOGICAL_REPAIR(child, precedence_graph)

```

---

```

19:   end if
20:   return child
21: end function

```

---

### Team Assignment Crossover (TAX)

The goal of **Team Assignment Crossover** is to:

- Inherit good cost-efficient or feasible team assignments from both parents
- Create diversity in the population
- Potentially reduce total cost or improve task feasibility

At each step, randomly choose the team assignment from one of the two parents:

```
child_team[i] ← random_choice([parent1_team[i], parent2_team[i]])
```

More specifically, for each task in the child's order, the team is chosen randomly from the two parent assignments identified by task identity. If neither parent provides a valid team, a random capable team is assigned.

---

#### Algorithm 15 Team Assignment Crossover (TAX)

---

```

1: function TAX(task_order1, team1, task_order2, team2, available_team,
   child_task_order)
2:   map1 ← {task : team | (task, team) ∈ zip(task_order1, team1)}
3:   map2 ← {task : team | (task, team) ∈ zip(task_order2, team2)}
4:   child_team ← []
5:   for currentTask ∈ child_task_order do
6:     possible ← []
7:     if currentTask ∈ map1 and map1[currentTask] ∈ available_team[currentTask]
8:       then
9:         possible.append(map1[currentTask])
10:      end if
11:     if currentTask ∈ map2 and map2[currentTask] ∈ available_team[currentTask]
12:       then
13:         possible.append(map2[currentTask])
14:       end if
15:     if |possible| > 0 then
16:       child_team.append(random choice from possible)
17:     else
18:       child_team.append(random choice from available_team[currentTask])
19:     end if
20:   end for

```

```

19:   return child_team
20: end function

```

---

### Offspring Generation

Two children are produced per crossover event by applying POX and TAX symmetrically on both parents.

---

#### Algorithm 16 Offspring Generation

---

```

1: function OFFSPRING_GENERATE(parent1, parent2, precedence, precedence_graph,
   available_team)
2:   to1, tm1 ← parent1.task_order, parent1.team_assignment
3:   to2, tm2 ← parent2.task_order, parent2.team_assignment
4:   child_task1 ← POX(to1, to2, precedence, precedence_graph)
5:   child_task2 ← POX(to2, to1, precedence, precedence_graph)
6:   child_team1 ← TAX(to1, tm1, to2, tm2, available_team, child_task1)
7:   child_team2 ← TAX(to2, tm2, to1, tm1, available_team, child_task2)
8:   return Chromosome(child_task1, child_team1), Chromosome(child_task2,
   child_team2)
9: end function

```

---

### 5.2.6 Mutation

Mutation introduces small, random changes to the offspring. For example:

- Changing the team assigned to a task
- Swapping two tasks in the order, if the new order is still valid

This improves the search in many ways:

1. Maintains genetic diversity
2. Avoids local optima
3. Explores the search space more thoroughly
4. Improves robustness

Two main mutations are used in this project:

- **Task Order Mutation:** Swap two tasks in the task order. The mutation is accepted only if the resulting order still satisfies all precedence constraints.



- **Team Assignment Mutation:** Randomly reassign a task to another valid team that is capable of performing that task.

---

**Algorithm 17** Mutation

---

```

1: function MUTATE(ind, precedence, available_team, attempts, mutation_rate)
2:   task_order, team_assignment  $\leftarrow$  ind.task_order[:], ind.team_assignment[:]
3:   new_task_order  $\leftarrow$  TASK_ORDER_MUTATION(task_order, team_assignment,
   precedence, attempts)
4:   new_team_assignment  $\leftarrow$  TEAM_ASSIGNMENT_MUTATION(new_task_order,
   team_assignment, available_team, mutation_rate)
5:   return Chromosome(new_task_order, new_team_assignment)
6: end function

```

---

### 5.2.7 Evaluate and Replace

This is when we evaluate the new offspring and replace them into the population. This step flow is as follows:

1. Append the offspring to the current population.
2. Evaluate and sort the population according to the lexicographic fitness 5.1.4:
3. Only keep the first `population_size` individuals of the population.

---

**Algorithm 18** Evaluate and Replace

---

```

1: function EVALUATE_AND_REPLACE(population, child1, child2, ...)
2:   population_size  $\leftarrow$  |population|
3:   population.extend([child1, child2])
4:   Sort population ascending by CALC_FITNESS(·)
5:   return population[: population_size]
6: end function

```

---

### 5.2.8 Termination Condition

The algorithm terminates when the elapsed wall-clock time reaches `max_time = 295` seconds.

### 5.2.9 Main Loop

---

**Algorithm 19** Genetic Algorithm — Main Loop

---

```
1: function GA_MAIN
2:   start  $\leftarrow$  time.time();   max_time  $\leftarrow$  295;   population_size  $\leftarrow$  10
3:   population  $\leftarrow$  INITIALIZE_POPULATION(population_size, ...)
4:   while True do
5:     if time.time() - start  $\geq$  max_time then
6:       break
7:     end if
8:     parent1, parent2  $\leftarrow$  RANK_SELECTION(population, ...)
9:     child1, child2  $\leftarrow$  OFFSPRING_GENERATE(parent1, parent2, ...)
10:    child1  $\leftarrow$  MUTATE(child1, precedence, available_team, 5, 0.05)
11:    child2  $\leftarrow$  MUTATE(child2, precedence, available_team, 5, 0.05)
12:    population  $\leftarrow$  EVALUATE_AND_REPLACE(population, child1, child2, ...)
13:  end while
14:  return population[0]
15: end function
```

---

## 5.3 Simulated Annealing

**Introduction:** Simulated Annealing (SA) is a meta-heuristic inspired by the physical annealing process in metallurgy, where a material is heated and then slowly cooled to reduce defects and reach a stable low-energy state.

**Main Idea:** This method balances exploration (accepting bad moves early on) and exploitation (refining good solutions later) by controlling a *temperature* parameter over time. In optimization, SA searches for a good solution by:

- Starting from a random solution
- Making small random changes, called perturbations
- Accepting changes that improve the solution
- Occasionally accepting worse solutions to escape local optima
- Gradually reducing the chance of accepting worse solutions by cooling

Since SA shares the solution representation, greedy initialization, decode, fitness, and neighbor generation components with GA (Section 5.1), only the SA-specific components are described below.

### 5.3.1 State Representation

The state representation of SA is identical to the solution representation defined in Section 5.1.2 (i.e. the same chromosome structure used by GA).

```

State: {
    "task_order"      : [3, 1, 4, 2, ..., 5],
    "team_assignment" : [2, 1, 3, 1, ..., 2]
}

```

Unlike GA, SA works with a single current state at a time rather than a population.

### 5.3.2 Create Initial Feasible Solution

SA starts from the greedy solution produced by `GREEDY_CONSTRUCT` defined in Section 5.1.3.

### 5.3.3 Decode State and Fitness Function

Identical to Section 5.1.4. The same lexicographic fitness is used:

$$\text{fitness} = (\text{num\_unscheduled\_task}, \text{completion\_time}, \text{total\_cost})$$

### 5.3.4 Neighbor Generation

A neighbor is produced by applying `TASK_ORDER_MUTATION` and `TEAM_ASSIGNMENT_MUTATION` in sequence, both defined in Section 5.1.6.

---

#### Algorithm 20 Get Neighbor (SA)

---

```

1: function GET_NEIGHBOR(curr, precedence, available_team, attempts,
   mutation_rate)
2:   task_order, team_assignment ← curr.task_order[:], curr.team_assignment[:]
3:   new_task_order ← TASK_ORDER_MUTATION(task_order, team_assignment,
   precedence, attempts)
4:   new_team_assignment ← TEAM_ASSIGNMENT_MUTATION(new_task_order,
   team_assignment, available_team, mutation_rate)
5:   return State(new_task_order, new_team_assignment)
6: end function

```

---

### 5.3.5 Acceptance Probability

The acceptance rule has two cases:

1. If the neighbor solution is no worse than the current solution:

$$f_{\text{neigh}} \leq f_{\text{curr}}$$

then the neighbor is always accepted.

2. Otherwise, the neighbor is worse. The algorithm finds the first objective where the two fitness tuples differ, then computes:

$$\Delta = \frac{b - a}{\max(1.0, |a|)}$$

where  $a$  is the current objective value and  $b$  is the neighbor objective value. The division by  $\max(1.0, |a|)$  normalizes the increase because different objectives have different units and scales.

The worse neighbor is accepted with probability:

$$p = \exp\left(-\frac{\Delta}{T}\right)$$

A high temperature  $T$  makes worse solutions easier to accept, while a low temperature makes them harder to accept.

---

**Algorithm 21** Acceptance Criterion (Metropolis)
 

---

```

1: function ACCEPTANCE( $f_{\text{curr}}, f_{\text{neigh}}, T$ )
2:   if  $f_{\text{neigh}} \leq f_{\text{curr}}$  then
3:     return True
4:   end if
5:   for  $(a, b) \in \text{zip}(f_{\text{curr}}, f_{\text{neigh}})$  do
6:     if  $a \neq b$  then
7:        $\Delta \leftarrow \frac{b - a}{\max(1.0, |a|)}$ ; break
8:     end if
9:   end for
10:   $p \leftarrow \exp\left(-\frac{\Delta}{T}\right)$ 
11:  return  $\text{random}() \leq p$ 
12: end function
    
```

---

### 5.3.6 Termination Condition

Identical to GA: the algorithm stops when elapsed time reaches  $\text{max\_time} = 295$  seconds.

### 5.3.7 Cooling Schedule and Main Loop

A linear cooling schedule is used:

$$T(t) = \max\left(T_{\min}, T_0 \times \left(1 - \frac{t}{\text{max\_time}}\right)\right)$$

where  $T_0 = 50.0$  and  $T_{\min} = 10^{-3}$ . The temperature is recomputed from elapsed time at every iteration, so it decreases monotonically without needing an explicit multiplicative step.

---

**Algorithm 22** Simulated Annealing — Main Loop

---

```

1: function SA_MAIN
2:   start  $\leftarrow$  time.time();   max_time  $\leftarrow$  295;    $T_0 \leftarrow 50.0$ ;    $T_{\min} \leftarrow 10^{-3}$ 
3:   curr  $\leftarrow$  GREEDY_CONSTRUCT(...)
4:   best  $\leftarrow$  curr;    $f_{\text{curr}} \leftarrow$  CALC_FITNESS(curr, ...);    $f_{\text{best}} \leftarrow f_{\text{curr}}$ 
5:   while True do
6:     elapsed  $\leftarrow$  time.time() – start
7:     if elapsed  $\geq$  max_time then
8:       break
9:     end if
10:     $T \leftarrow \max(T_{\min}, T_0 \times (1 - \text{elapsed}/\text{max\_time}))$ 
11:    neighbor  $\leftarrow$  GET_NEIGHBOR(curr, precedence, available_team, 5, 0.1)
12:     $f_{\text{neigh}} \leftarrow$  CALC_FITNESS(neighbor, ...)
13:    if ACCEPTANCE( $f_{\text{curr}}, f_{\text{neigh}}, T$ ) then
14:      curr,  $f_{\text{curr}} \leftarrow$  neighbor,  $f_{\text{neigh}}$ 
15:      if  $f_{\text{neigh}} < f_{\text{best}}$  then
16:        best,  $f_{\text{best}} \leftarrow$  neighbor,  $f_{\text{neigh}}$ 
17:      end if
18:    end if
19:  end while
20:  return best
21: end function

```

---

## 5.4 Tabu Search

**Introduction:** Tabu Search (TS) enhances local search by incorporating a memory structure *the tabu list* to prevent cycling and facilitate escape from local optima. At each iteration, the best admissible neighbor is selected, even if it is worse than the current solution.

**Main Idea:** TS inherits the greedy selection of hill climbing by always choosing the best neighbor from a generated neighbor list. The difference lies in the tabu list, which stores recent moves to prevent immediately undoing them:

- The tabu list records recently performed moves, marking them as temporarily forbidden.
- The duration a move stays forbidden is called the *tabu tenure* (measured in iterations).
- This helps the algorithm avoid cycling, escape local optima, and maintain search diversity.

### 5.4.1 State and Move Representation

The state representation of Tabu Search is identical to the solution representation defined in Section 5.1.2. Therefore, each state still consists of a `task_order` and a `team_assignment`.

The main difference is that Tabu Search additionally records the move used to generate a neighbor. In this implementation, a move is represented as:

$$\text{move} = (\text{swap\_key}, \text{team\_changes})$$

where:

- `swap_key` records the pair of swapped tasks:

$$\text{swap\_key} = \text{tuple}(\text{sorted}((\text{task\_i}, \text{task\_j})))$$

If no valid swap is performed, `swap_key = None`.

- `team_changes` records all team reassignment changes:

$$\text{team\_changes} = ((\text{task}, \text{old\_team}, \text{new\_team}), \dots)$$

If no team assignment is changed, it is an empty tuple.

This representation is compact because the tabu list stores only recent moves, not entire solutions.

### 5.4.2 Create Initial Feasible State

Identical to SA: the search starts from the greedy solution produced by `GREEDY_CONSTRUCT` define in Section 5.1.3.

### 5.4.3 Decode State and Fitness Calculation

Identical to Section 5.1.4.

### 5.4.4 Neighbor Generation

The TS variant of the neighbor generation extends the shared mutation operators to also return move-tracking information required by the tabu mechanism.

---

**Algorithm 23** Task Order Mutation with Move Tracking (TS)

---

---

```

1: function TASK_ORDER_MUTATION_TS(task_order, team_assignment,
   precedence, attempts)
2:   if |task_order| < 2 then
3:     return task_order[:, team_assignment[:, None
4:   end if
5:   for  $k \leftarrow 1$  to attempts do
6:     new_order  $\leftarrow$  task_order[:,
7:      $i, j \leftarrow$  random sample of 2 distinct indices
8:     swap(new_order[i], new_order[j])
9:     if IS_PRECEDENCE_FEASIBLE(new_order, precedence) then
10:      new_team  $\leftarrow$  team_assignment[:,
11:      swap(new_team[i], new_team[j])
12:      swap_key  $\leftarrow$  tuple(sorted({task_order[i], task_order[j]}))
13:      return new_order, new_team, swap_key
14:     end if
15:   end for
16:   return task_order[:, team_assignment[:, None
17: end function

```

---



---

**Algorithm 24** Team Assignment Mutation with Change Tracking (TS)

---

```

1: function TEAM_ASSIGNMENT_MUTATION_TS(task_order, team_assignment,
   available_team, mutation_rate)
2:   new_assignment  $\leftarrow$  team_assignment[:, changes  $\leftarrow$  []
3:   for (idx, task)  $\in$  enumerate(task_order) do
4:     if random() < mutation_rate then
5:       viable  $\leftarrow$  available_team[task]
6:       if |viable| > 1 then
7:         new_team  $\leftarrow$  random choice from viable
8:         if new_team  $\neq$  new_assignment[idx] then
9:           changes.append((task, new_assignment[idx], new_team))
10:          new_assignment[idx]  $\leftarrow$  new_team
11:         end if
12:       end if
13:     end if
14:   end for
15:   return new_assignment, changes
16: end function

```

---



---

**Algorithm 25** Get Neighbor with Move (TS)

---

```

1: function GET_NEIGHBOR_TS(curr, precedence, available_team, attempts,
   mutation_rate)
2:   new_to, team_swap,
   swap_key  $\leftarrow$  TASK_ORDER_MUTATION_TS(curr.task_order,
   curr.team_assignment, precedence, attempts)

```

---

```

3:   new_tm, team_changes ← TEAM_ASSIGNMENT_MUTATION_TS(new_to,
   team_swap, available_team, mutation_rate)
4:   move ← (swap_key, tuple(team_changes))
5:   return State(new_to, new_tm), move
6: end function

```

---

### 5.4.5 Move Identification and Tabu Check

This is the main difference of Tabu Search. After a neighbor is generated, the algorithm identifies the move that produced it and checks whether the move is admissible.

A move is considered **admissible** in one of two cases:

- **Not tabu:** the move is not in the tabu list, so applying it does not immediately cycle back to a recently visited solution.
- **Aspiration criterion:** even if the move is tabu, it is still allowed if the resulting solution improves the current global best solution.

Therefore, the tabu list prevents cycling, while the aspiration criterion allows the search to accept a very good move even when it is temporarily tabu.

---

#### Algorithm 26 Tabu Check

---

```

1: function IS_TABU(move, tabu_swap, tabu_team, gen)
2:   swap_key, team_changes ← move
3:   if swap_key ≠ None then
4:     expire ← tabu_swap.get(swap_key)
5:     if expire ≠ None and expire > gen then
6:       return True
7:     end if
8:   end if
9:   for (task, old_team, new_team) ∈ team_changes do
10:    expire ← tabu_team.get((task, new_team))
11:    if expire ≠ None and expire > gen then
12:      return True
13:    end if
14:   end for
15:   return False
16: end function

```

---



---

#### Algorithm 27 Tabu Registration

---

```

1: function REGISTER_TABU(move, tabu_swap, tabu_team, gen, tenure)
2:   swap_key, team_changes ← move
3:   if swap_key ≠ None then

```



```

4:     tabu_swap[swap_key]  $\leftarrow$  gen + tenure
5:   end if
6:   for (task, old_team, new_team)  $\in$  team_changes do
7:     tabu_team[(task, old_team)]  $\leftarrow$  gen + tenure
8:   end for
9: end function

```

---

### 5.4.6 Termination Condition

The algorithm terminates when elapsed time reaches  $\text{max\_time} = 295$  seconds. Expired tabu entries are pruned every 200 generations to manage memory.

### 5.4.7 Main Loop

---

#### Algorithm 28 Tabu Search — Main Loop

---

```

1: function TS_MAIN
2:
   start  $\leftarrow$  time.time(); max_time  $\leftarrow$  295; num_neighbors  $\leftarrow$  10; tabu_tenure  $\leftarrow$  10
3:   tabu_swap  $\leftarrow$  {}; tabu_team  $\leftarrow$  {}
4:   curr  $\leftarrow$  GREEDY_CONSTRUCT(...);  $f_{\text{curr}} \leftarrow$  CALC_FITNESS(curr, ...)
5:   best  $\leftarrow$  curr;  $f_{\text{best}} \leftarrow f_{\text{curr}}$ ; gen  $\leftarrow$  0
6:   while True do
7:     if time.time() - start  $\geq$  max_time then
8:       break
9:     end if
10:    gen += 1; best_candidate  $\leftarrow$  None;  $f_{\text{bc}} \leftarrow$  None; best_move  $\leftarrow$  None
11:    for _  $\leftarrow$  1 to num_neighbors do
12:      neighbor, move  $\leftarrow$  GET_NEIGHBOR_TS(curr, precedence,
available_team, 5, 0.1)
13:      fit  $\leftarrow$  CALC_FITNESS(neighbor, ...)
14:
admissible  $\leftarrow$   $\neg$ IS_TABU(move, tabu_swap, tabu_team, gen) or (fit <  $f_{\text{best}}$ )
15:      if admissible and ( $f_{\text{bc}} = \text{None}$  or fit <  $f_{\text{bc}}$ ) then
16:        best_candidate,  $f_{\text{bc}}$ , best_move  $\leftarrow$  neighbor, fit, move
17:      end if
18:    end for
19:    if best_candidate = None then
20:      continue
21:    end if
22:    curr,  $f_{\text{curr}} \leftarrow$  best_candidate,  $f_{\text{bc}}$ 
23:    if  $f_{\text{curr}} < f_{\text{best}}$  then
24:      best,  $f_{\text{best}} \leftarrow$  curr,  $f_{\text{curr}}$ 

```

```
25:     end if
26:     REGISTER_TABU(best_move, tabu_swap, tabu_team, gen, tabu_tenure)
27:     if gen mod 200 = 0 then
28:         Remove entries from tabu_swap and tabu_team where expire  $\leq$  gen
29:     end if
30: end while
31: return best
32: end function
```

---

## 5.5 Ant Colony Optimization

**Introduction:** Ant Colony Optimization (ACO) is a population-based metaheuristic inspired by the foraging behavior of real ants. In nature, ants deposit pheromone on paths leading to food sources. Paths with stronger pheromone are more likely to be followed by later ants, so good paths are gradually reinforced over time.

**Main Idea:** In this project, each ant constructs a solution by assigning a feasible team to each task. The team choice is probabilistic and depends on two main factors: *pheromone intensity*, which represents learned experience from previous good solutions, and *heuristic desirability*, which estimates how promising a team assignment is at the current step.

The general procedure of ACO is:

1. Initialize pheromone values for all valid task–team assignments.
2. Let multiple ants construct candidate solutions.
3. Evaluate all constructed solutions using the common fitness function.
4. Evaporate pheromone globally and reinforce good assignments.
5. Repeat until the time limit is reached.

### 5.5.1 State Representation

ACO uses the same state representation as the other metaheuristic methods. As defined in Section 5.1.2, each solution consists of two components:

$$\text{solution} = (\text{task\_order}, \text{team\_assignment})$$

In ACO, each ant builds one such solution. The `task_order` is the common schedulable order, while `team_assignment` is constructed step by step according to pheromone and heuristic information.

### 5.5.2 Create Initial Feasible Solution

ACO uses the same greedy construction procedure described in Section 5.1.3 to create an initial feasible solution.

This greedy solution is used as a seed for the search. It provides an initial best solution before the ants start exploring alternative assignments.

$$\text{best\_solution} \leftarrow \text{Greedy\_Construct}()$$

After that, ACO improves the solution by repeatedly generating new candidate solutions through ant construction and pheromone updates.

### 5.5.3 Decode State and Fitness Function

ACO also uses the same decoding and fitness evaluation mechanism described in Section 5.1.4.

Therefore, ACO does not define a new fitness function. Its main difference lies in how new team assignments are constructed and how pheromone values are updated.

### 5.5.4 Pheromone Representation and Initialization

Pheromone is stored as a nested dictionary  $\tau[\text{task}][\text{team}]$ , representing the learned desirability of assigning **team** to **task**. All entries are initialised to  $\tau_0 = 1.0$ . The pheromone trails are bounded between  $\tau_{\min}$  and  $\tau_{\max}$  following the Max–Min Ant System (MMAS) variant.

---

#### Algorithm 29 Pheromone Initialization

---

```

1: function INIT_PHEROMONE(schedulable_order, available_team)
2:    $\tau \leftarrow \{\}$ 
3:   for task  $\in$  schedulable_order do
4:      $\tau[\text{task}] \leftarrow \{\text{team} : \tau_0 \mid \text{team} \in \text{available\_team}[\text{task}]\}$ 
5:   end for
6:   return  $\tau$ 
7: end function

```

---

### 5.5.5 Ant Solution Construction

This is the core step of Ant Colony Optimization. Each ant constructs one complete solution by processing tasks in the fixed schedulable order. At each task, the ant chooses one feasible team using **roulette-wheel selection**.

For a current task, each candidate team is assigned a selection weight:

$$w(\text{task}, \text{team}) = \tau[\text{task}][\text{team}]^\alpha \times \eta(\text{task}, \text{team})^\beta$$

where:

- $\tau[\text{task}][\text{team}]$  is the pheromone value of assigning that team to the task. A larger pheromone value means this assignment has appeared in good previous solutions.
- $\eta(\text{task}, \text{team})$  is the heuristic desirability of the assignment:

$$\eta(\text{task}, \text{team}) = \frac{1}{\text{estimated finish time} + 1}$$

A team with an earlier estimated finish time has a larger heuristic value.

- $\alpha$  controls the influence of pheromone information.
- $\beta$  controls the influence of heuristic information.

After computing all candidate weights, the probability of choosing a team is:

$$P(\text{team}) = \frac{w(\text{task}, \text{team})}{\sum_{k \in \text{candidate teams}} w(\text{task}, k)}$$

Therefore, roulette-wheel selection works as follows:

- teams with larger weights occupy larger portions of the roulette wheel;
- a random value is sampled from the total weight;
- the selected team is the first one whose cumulative weight reaches that random value.

This mechanism balances two goals: exploiting assignments with strong pheromone trails and exploring assignments that look promising according to the current schedule.

---

### Algorithm 30 Ant Solution Construction

---

```

1: function ANT_CONSTRUCT(schedulable_order, rev_graph, available_team,
   duration, team_avail,  $\tau$ ,  $\alpha$ ,  $\beta$ )
2:   task_End  $\leftarrow$   $[0] \times \text{num\_task}$ ; teamAvail  $\leftarrow$  team_avail[:]
3:   task_order  $\leftarrow$  []; team_assignment  $\leftarrow$  []
4:   for currentTask  $\in$  schedulable_order do
5:     pred_finish  $\leftarrow$  max(task_End[p] |  $p \in \text{rev\_graph}[\text{currentTask}]$ )
6:     teams  $\leftarrow$  available_team[currentTask]; weights  $\leftarrow$  []
7:     for team  $\in$  teams do
8:       earliest  $\leftarrow$  max(teamAvail[team], pred_finish)
```

```

9:         finish  $\leftarrow$  earliest + duration[currentTask]
10:         $\eta \leftarrow 1.0/(\text{finish} + 1)$ 
11:        weights.append( $\tau[\text{currentTask}][\text{team}]^\alpha \times \eta^\beta$ )
12:    end for
13:    total  $\leftarrow \sum$  weights
14:    if total = 0 then
15:        chosen  $\leftarrow$  random choice from teams
16:    else
17:         $r \leftarrow \text{random}() \times \text{total}$ ;    cumulative  $\leftarrow 0$ ;    chosen  $\leftarrow \text{teams}[-1]$ 
18:        for (team, w)  $\in$  zip(teams, weights) do
19:            cumulative += w
20:            if cumulative  $\geq r$  then
21:                chosen  $\leftarrow$  team;    break
22:            end if
23:        end for
24:    end if
25:    earliest  $\leftarrow \max(\text{teamAvail}[\text{chosen}], \text{pred\_finish})$ 
26:    task_End[currentTask]  $\leftarrow$  earliest + duration[currentTask]
27:    teamAvail[chosen]  $\leftarrow$  task_End[currentTask]
28:    task_order.append(currentTask);    team_assignment.append(chosen)
29: end for
30: return Solution(task_order, team_assignment)
31: end function

```

---

### 5.5.6 Pheromone Update

After each iteration, ACO updates pheromone values in two main steps:

- **Evaporation:** decrease all pheromone values to reduce the influence of old search information.
- **Deposit:** add pheromone to good task–team assignments found by the global best and the iteration best solutions.

All pheromone values are kept within the range  $[\tau_{\min}, \tau_{\max}]$  to avoid values becoming too small or too large.

#### Evaporation

- Every pheromone value is multiplied by  $(1 - \rho)$ .
- The lower bound  $\tau_{\min}$  ensures that each valid assignment still has a chance to be selected in later iterations.

**Algorithm 31** Pheromone Evaporation

---

```

1: function EVAPORATE( $\tau$ ,  $\rho$ )
2:   for task  $\in \tau$  do
3:     for team  $\in \tau[\text{task}]$  do
4:        $\tau[\text{task}][\text{team}] \leftarrow \max(\tau_{\min}, \tau[\text{task}][\text{team}] \times (1 - \rho))$ 
5:     end for
6:   end for
7: end function

```

---

**Deposit**

- Good solutions deposit additional pheromone on the assignments they use.
  - A better solution receives a larger pheromone contribution.
  - The upper bound  $\tau_{\max}$  prevents one assignment from dominating the search too early.
- 

**Algorithm 32** Pheromone Deposit

---

```

1: function DEPOSIT( $\tau$ , sol, fitness)
2:    $(\cdot, \text{completion}, \cdot) \leftarrow \text{fitness}$ 
3:    $\delta \leftarrow 1.0/(\text{completion} + 1)$ 
4:   for (task, team)  $\in \text{zip}(\text{sol.task\_order}, \text{sol.team\_assignment})$  do
5:      $\tau[\text{task}][\text{team}] \leftarrow \min(\tau_{\max}, \tau[\text{task}][\text{team}] + \delta)$ 
6:   end for
7: end function

```

---

**5.5.7 Termination Condition**

The algorithm terminates when elapsed time reaches  $\text{max\_time} = 295$  seconds.

**5.5.8 Main Loop**

ACO parameters:  $\text{num\_ants} = 10$ ,  $\alpha = 1.0$ ,  $\beta = 2.0$ ,  $\rho = 0.1$ ,  $\tau_0 = 1.0$ ,  $\tau_{\min} = 10^{-4}$ ,  $\tau_{\max} = 10.0$ . The global best and iteration best both deposit pheromone each round to balance exploitation and exploration.

---

**Algorithm 33** Ant Colony Optimization — Main Loop

---

```

1: function ACO_MAIN
2:   start  $\leftarrow \text{time.time}()$ ;   max_time  $\leftarrow 295$ 
3:    $\tau \leftarrow \text{INIT\_PHEROMONE}(\text{schedulable\_order}, \text{available\_team})$ 
4:   best  $\leftarrow \text{GREEDY\_CONSTRUCT}(\dots)$ ;    $f_{\text{best}} \leftarrow \text{CALC\_FITNESS}(\text{best}, \dots)$ 
5:   DEPOSIT( $\tau$ , best,  $f_{\text{best}}$ )

```

---

---

```

6:   while True do
7:     elapsed  $\leftarrow$  time.time() - start
8:     if elapsed  $\geq$  max_time then
9:       break
10:    end if
11:    iter_best  $\leftarrow$  None;  f_iter  $\leftarrow$  None
12:    for _  $\leftarrow$  1 to num_ants do
13:      ant_sol  $\leftarrow$  ANT_CONSTRUCT(...,  $\tau$ ,  $\alpha$ ,  $\beta$ )
14:      f_ant  $\leftarrow$  CALC_FITNESS(ant_sol, ...)
15:      if f_iter = None or f_ant < f_iter then
16:        iter_best, f_iter  $\leftarrow$  ant_sol, f_ant
17:      end if
18:      if f_ant < f_best then
19:        best, f_best  $\leftarrow$  ant_sol, f_ant
20:      end if
21:    end for
22:    EVAPORATE( $\tau$ ,  $\rho$ )
23:    DEPOSIT( $\tau$ , best, f_best)
24:    if iter_best  $\neq$  None then
25:      DEPOSIT( $\tau$ , iter_best, f_iter)
26:    end if
27:  end while
28:  return best
29: end function

```

---

## 5.6 Results

This section compares the judge scores obtained by the four implemented metaheuristic algorithms: Genetic Algorithm (GA), Ant Colony Optimization (ACO), Simulated Annealing (SA), and Tabu Search (TS).

Each result is reported as:

(tasks scheduled, completion time, total cost)

This corresponds to the fitness criteria in Section 5.1.4. For readability, the table shows the number of scheduled tasks directly, while the internal fitness minimizes the number of unscheduled tasks.

**Comparison rule.** All algorithms are compared lexicographically:

more scheduled tasks  $\rightarrow$  smaller completion time  $\rightarrow$  lower total cost.

In this dataset, all methods schedule the same number of tasks as the jury, so completion time and total cost become the main differentiators.

### 5.6.1 Score Comparison

Table 5.1: Jury reference score versus GA, ACO, SA, and TS on ten test cases. Each cell is written as (tasks scheduled, completion time, total cost). Bold entries indicate the best algorithmic result in each row; ties are bolded together.

| No. | Tasks | Teams | Prec. | Jury              | GA                      | ACO                      | SA                       | TS                    |
|-----|-------|-------|-------|-------------------|-------------------------|--------------------------|--------------------------|-----------------------|
| 1   | 5     | 6     | 4     | (5, 235, 145)     | <b>(5, 235, 145)</b>    | <b>(5, 235, 145)</b>     | <b>(5, 235, 145)</b>     | <b>(5, 235, 145)</b>  |
| 2   | 10    | 15    | 6     | (10, 210, 430)    | (10, 210, 350)          | (10, 190, 530)           | <b>(10, 190, 480)</b>    | <b>(10, 190, 480)</b> |
| 3   | 100   | 80    | 100   | (100, 550, 4510)  | <b>(100, 550, 1920)</b> | (100, 550, 2090)         | (100, 550, 3050)         | (100, 550, 4510)      |
| 4   | 200   | 80    | 150   | (200, 370, 13240) | (200, 360, 8880)        | (200, 360, 7140)         | <b>(200, 350, 14950)</b> | (200, 370, 13240)     |
| 5   | 500   | 200   | 650   | (493, 840, 38150) | (493, 780, 32430)       | <b>(493, 780, 28680)</b> | (493, 780, 40210)        | (493, 780, 39930)     |
| 6   | 600   | 300   | 750   | (599, 550, 42300) | (599, 530, 41210)       | <b>(599, 530, 32210)</b> | (599, 530, 55540)        | (599, 550, 42300)     |
| 7   | 700   | 300   | 750   | (657, 510, 46270) | (657, 450, 50060)       | <b>(657, 450, 34800)</b> | (657, 490, 63890)        | (657, 510, 46270)     |
| 8   | 800   | 400   | 750   | (786, 530, 63960) | (786, 520, 60040)       | <b>(786, 520, 50330)</b> | (786, 520, 72680)        | (786, 520, 81880)     |
| 9   | 900   | 400   | 750   | (884, 820, 74300) | (884, 710, 66090)       | <b>(884, 710, 58660)</b> | (884, 710, 77510)        | (884, 710, 83740)     |
| 10  | 1000  | 400   | 1750  | (980, 770, 68520) | (980, 760, 72510)       | <b>(980, 760, 62150)</b> | (980, 760, 95070)        | (980, 770, 68520)     |

*Note.* All four algorithms use the same wall-clock budget of approximately 295 seconds. Therefore, runtime is not shown in the table because it does not distinguish the methods.

### 5.6.2 Observations

- **All methods match the jury in scheduled-task count.** This means the search mainly affects completion time and total cost, rather than the number of feasible tasks.
- **Small instances are easy for all methods.** On the smallest test case, all algorithms obtain exactly the same score. On the second test case, SA and TS even improve the jury completion time from 210 to 190.
- **ACO performs best on large instances.** From  $N = 500$  to  $N = 1000$ , ACO gives the best result in every test case. Its advantage mainly comes from lower total cost while keeping the same completion time as the other strong methods.
- **GA is competitive on medium-sized cases.** GA achieves the best result for  $N = 100$  and remains close to ACO in several larger cases, but it is usually more expensive than ACO.
- **SA and TS are strong in completion time but weaker in cost.** These methods often reach good makespan values, but their team assignments tend to have higher total cost on larger instances.
- **Overall, ACO is the most stable method in this experiment.** Since ACO focuses directly on task–team assignment through pheromone learning, it exploits cost-efficient assignments better under the same time limit.



# Chapter 6

## Additional Algorithms

### 6.1 Iterated Local Search

**Introduction:** Iterated Local Search (ILS) is a metaheuristic optimization method. While local search methods like simple Hill Climbing get trapped in low-quality local minima where no further improving neighbor is available, ILS systematically escapes these traps by alternating between an intense local search and a structured diversification step. Unlike Simulated Annealing (SA), which accepts degrading moves probabilistically, or Tabu Search (TS), which utilizes memory structures to avoid cyclic behavior, ILS guides the search straight through the subspace of local optima. By focusing the exploration on local minima rather than the entire feasible search space, ILS achieves top-tier performance with minimal parameter tuning and fast execution.

**Main Idea:** High-quality local minima are closer to other high quality local minima than to random states. Finding a global optimal or near-optimal solution is more effective when starting the search from a high quality local minimum rather than restarting the search from a random configuration. ILS builds a randomized walk in the space of local optima by iteratively applying four core components:

1. **Initialization:** Generating a starting feasible solution and applying local search to reach the first local minimum.
2. **Perturbation:** Applying a randomized modification to the current local minimum to escape the current attraction basin.
3. **Subsidiary Local Search:** Refining the perturbed solution to obtain an alternative local minimum.
4. **Acceptance Criterion:** Determining whether the newly discovered local minimum should replace the current solution as the starting point for the next iteration.

### 6.1.1 State Representation

To ensure integration within the existing scheduling platform, the state representation of the ILS candidate solution matches the structure utilized in Simulated Annealing and Tabu Search. A candidate schedule state  $s$  consists of a topological task ordering sequence and a corresponding team assignment mapping:

$$s = \{\text{"task\_order"} : T, \text{"team\_assignment"} : A\}$$

Where:

- $T = [t_1, t_2, \dots, t_M]$  represents a topologically valid order of the  $M$  project tasks, satisfying the precedence constraints defined in  $Q$ .
- $A = [a_1, a_2, \dots, a_M]$  is an array where  $a_i \in \{1, 2, \dots, N\}$  denotes the specific team assigned to execute task  $t_i$ . Each assignment  $a_i$  must belong to the subset of teams capable of performing task  $t_i$ .

### 6.1.2 Create Initial Feasible State

Similar with SA and TS, the Initial State is also where ILS starts exploring, and the initialization of ILS is exactly the same with SA (utilizing the greedy construction method).

### 6.1.3 Decode State and Fitness Calculation

Again, similar with SA and TS.

### 6.1.4 Perturbation Mechanism

The perturbation mechanism is the most critical operator in an ILS algorithm. Its primary goal is to escape the current local minimum's attraction basin. If the perturbation is too weak, the local search will undo the modification and turn back into the same local minimum. If the perturbation is too strong, the algorithm loses the structural information accumulated during the search.

The perturbation operator modifies the schedule in a way that cannot be directly reversed by a single step of the standard local search neighborhood. The perturbation strength is controlled by a parameter  $d$ , which defines the number of random moves applied. A hybrid perturbation strategy is utilized, executing a combination of  $d$  randomized sequence swaps and  $d$  resource reassignments:

1. **Task Sequence Perturbation:** Swapping two randomly selected non-adjacent tasks in the `task_order` list. A swap is only executed if it does not violate any precedence constraints  $Q$ .

2. **Team Assignment Perturbation:** Reassigning a randomly chosen task to an alternative qualified team, targeting the cost and resource allocation.

### Pseudo Code

```

1 Operation: Perturbation Step (Kick Move)
2 1: function PERTURB(state, d, precedence, available_team)
3 2:   new_state <- copy(state)
4 3:   for step from 1 to d do
5 4:     if random_boolean() then
6 5:       idx1 <- random_integer(0, length(state["task_order"])
7   - 1)
8 6:       idx2 <- random_integer(0, length(state["task_order"])
9   - 1)
10 7:       while idx1 == idx2 do
11 8:         idx2 <- random_integer(0, length(state["task_order
12   "]) - 1)
13 9:       end while
14 10:      swap new_state["task_order"][idx1], new_state["
15   task_order"][idx2]
16 11:      if not IS_PRECEDENCE_FEASIBLE(new_state["task_order
17   "], precedence) then
18 12:        revert swap
19 13:      end if
20 14:    else
21 15:      idx <- random_integer(0, length(state["task_order"])
22   - 1)
23 16:      task <- state["task_order"][idx]
24 17:      if length(available_team[task]) > 1 then
25 18:        current_team <- state["team_assignment"][idx]
26 19:        new_state["team_assignment"][idx] <-
27   random_choice(available_team[task] except current_team)
28 20:      end if
29 21:    end if
30 22:  end for
31 23:  return new_state
32 24: end function

```

### 6.1.5 Local Search via Variable Neighborhood Descent

The candidate state is perturbed, a subsidiary local search is executed to climb to the nearest local optimum. While a standard local search relies on a single neighborhood structure, this implementation integrates Variable Neighborhood Descent (VND). VND systematically searches through two larger neighborhood structures:

- **Neighborhood  $N_1$  (Adjacent Task Swap):** Swaps two adjacent tasks in the `task_order`

list without violating precedence.

- **Neighborhood  $N_2$  (Single Task Team Reassignment):** Reassigns a single task to an alternative qualified team to optimize assignment costs.

### Pseudo Code

```

1 Operation: Variable Neighborhood Descent (VND) Local Search
2 1: function VND_LOCAL_SEARCH(state, precedence, available_team, cost
   , duration)
3 2:   current_state <- state
4 3:   best_fit <- CALC_FITNESS(current_state, cost, duration)
5 4:   l <- 1
6 5:   l_max <- 2
7 6:   while l <= l_max do
8 7:     improved <- false
9 8:     if l == 1 then
10 9:       for idx in range(0, length(current_state["task_order
   "]) - 1) do
11 10:         neighbor <- copy(current_state)
12 11:         swap neighbor["task_order"][idx], neighbor["
   task_order"][idx + 1]
13 12:         if IS_PRECEDENCE_FEASIBLE(neighbor["task_order"],
   precedence) then
14 13:           fit <- CALC_FITNESS(neighbor, cost, duration)
15 14:           if fit < best_fit then
16 15:             current_state <- neighbor
17 16:             best_fit <- fit
18 17:             improved <- true
19 18:           end if
20 19:         end if
21 20:       end for
22 21:     else if l == 2 then
23 22:       for idx in range(0, length(current_state["task_order
   "])) do
24 23:         task <- current_state["task_order"][idx]
25 24:         current_team <- current_state["team_assignment"][
   idx]
26 25:         for team in available_team[task] do
27 26:           if team != current_team then
28 27:             neighbor <- copy(current_state)
29 28:             neighbor["team_assignment"][idx] <- team
30 29:             fit <- CALC_FITNESS(neighbor, cost,
   duration)
31 30:             if fit < best_fit then
32 31:               current_state <- neighbor
33 32:               best_fit <- fit
34 33:               improved <- true
35 34:             end if

```

```

36:         end if
37:     end for
38: end for
39: end if
40: if improved then
41:     l <- 1
42: else
43:     l <- l + 1
44: end if
45: end while
46: return current_state
47: end function

```

### 6.1.6 Acceptance Criterion

The acceptance criterion determines whether the newly obtained local optimum  $s''$  is accepted as the base solution for the next iteration ( $s_{curr} \leftarrow s''$ ), introducing a crucial bias between exploitation and exploration. Three primary acceptance strategies can be configured:

1. **Extreme Exploitation (Better Only):** The new local minimum is accepted only if it strictly improves upon the current incumbent. While this rapidly finds high-quality schedules in a specific region, it runs a high risk of getting trapped in a sub-optimal basin of attraction.
2. **Extreme Exploration(Random Walk):** The new local minimum is always accepted, regardless of its quality. This enables broad exploration but fails to exploit high-quality regions.
3. **Probabilistic Acceptance:** To balance these two extremes, a temperature-dependent probabilistic threshold is implemented. If the new local minimum  $s''$  is worse than the current  $s_{curr}$ , it is accepted with a probability:

$$P = \exp\left(-\frac{\Delta E}{T}\right)$$

Where:

- $\Delta E$  represents the normalized degradation in lexicographical fitness between  $f(s_{curr})$  and  $f(s'')$ .
- $T$  is the current temperature, which cools down at each iteration according to a geometric schedule ( $T \leftarrow T \times \alpha$ ).

This probabilistic threshold allows the search to wander across adjacent basins of attraction during early iterations when the temperature is high, while gradually narrowing its focus to concentrate around the optimal regions as the temperature cools.

## Pseudo Code

```

1 Operation: Probabilistic Acceptance Check
2 1: function ACCEPT_CANDIDATE(current_fit, candidate_fit, temperature
   )
3 2:   if candidate_fit < current_fit then
4 3:     return True
5 4:   end if
6 5:   delta <- normalize_lexicographical_degradation(current_fit,
   candidate_fit)
7 6:   prob <- exp(-delta / temperature)
8 7:   r <- random_number(0.0, 1.0)
9 8:   return r <= prob
10 9: end function

```

## 6.1.7 Main Loop

### Pseudo Code

```

1 Operation: Main ILS Loop
2 1: function MAIN_ILS
3 2:   start_time <- time.time()
4 3:   max_time <- 1.0
5 4:   max_iter <- 10000
6 5:   d <- 3
7 6:   Temp <- 100.0
8 7:   alpha <- 0.95
9 8:
10 9:   curr <- GREEDY_CONSTRUCT(num_task, precedence, graph,
   team_available_time, available_team, task_duration, cost)
11 10:
12 11:   curr_opt <- VND_LOCAL_SEARCH(curr, precedence, available_team
   , cost, task_duration)
13 12:   best <- curr_opt
14 13:   f_best <- CALC_FITNESS(best, cost, task_duration)
15 14:
16 15:   iter <- 0
17 16:   while iter < max_iter do
18 17:     if time.time() - start_time > max_time then
19 18:       break
20 19:     end if
21 20:
22 21:     perturbed <- PERTURB(curr_opt, d, precedence,
   available_team)
23 22:
24 23:     candidate_opt <- VND_LOCAL_SEARCH(perturbed, precedence,
   available_team, cost, task_duration)

```

```

25 24:         f_cand <- CALC_FITNESS(candidate_opt, cost, task_duration
26      )
27 25:         f_curr <- CALC_FITNESS(curr_opt, cost, task_duration)
28 26:
29 27:         if f_cand < f_best then
30 28:             best <- candidate_opt
31 29:             f_best <- f_cand
32 30:         end if
33 31:
34 32:         if ACCEPT_CANDIDATE(f_curr, f_cand, Temp) then
35 33:             curr_opt <- candidate_opt
36 34:         end if
37 35:
38 36:         Temp <- Temp * alpha
39 37:         iter <- iter + 1
40 38:     end while
41 39:     return best
42 40: end function

```

## 6.2 Variable Neighborhood Search

**Introduction:** Variable Neighborhood Search (VNS) is a metaheuristic optimization method. While Simulated Annealing (SA) accepts degrading moves probabilistically and Tabu Search (TS) relies on memory-based classification lists to avoid cyclic behavior, VNS utilizes a systematic change of neighborhood structures during the search process.

**Main idea:** The structural foundation of VNS relies on three critical observations:

1. A local minimum with respect to one neighborhood structure is not a local minimum for alternative neighborhood structures.
2. A global minimum is a local minimum with respect to all possible neighborhood structures.
3. Local minima relative to different neighborhoods are located close to each other.

### 6.2.1 State Representation

To ensure integration within the existing scheduling platform, the state representation of the VNS candidate solution matches the structure utilized in Simulated Annealing and Tabu Search. A candidate schedule state  $s$  consists of a topological task ordering sequence and a corresponding team assignment mapping:

$$s = \{\text{"task\_order"} : T, \text{"team\_assignment"} : A\}$$

Where:

- $T = [t_1, t_2, \dots, t_M]$  represents a topologically valid order of the  $M$  project tasks, satisfying the precedence constraints defined in  $Q$ .
- $A = [a_1, a_2, \dots, a_M]$  is an array where  $a_i \in \{1, 2, \dots, N\}$  denotes the specific team assigned to execute task  $t_i$ . Each assignment  $a_i$  must belong to the subset of teams capable of performing task  $t_i$ .

### 6.2.2 Create Initial Feasible State

Similar with SA and TS, the Initial State is also where VNS starts exploring, and the initialization of VNS is exactly the same with SA (utilizing the greedy construction method).

### 6.2.3 Decode State and Fitness Calculation

Again, similar with SA and TS.

### 6.2.4 Neighborhood Structures

Let  $\mathcal{N} = \{N_1, N_2, \dots, N_{k_{max}}\}$  be a set of predefined neighborhood structures, where the neighborhood sizes increase progressively:

$$\|N_1(s)\| < \|N_2(s)\| < \dots < \|N_{k_{max}}(s)\|$$

Three unique neighborhood structures ( $k_{max} = 3$ ) have been designed to target scheduling sequence optimization and resource allocation:

1. **Neighborhood  $N_1$  (Adjacent Task Swap):**  $N_1(s)$  contains all states generated by swapping two adjacent tasks  $t_i$  and  $t_{i+1}$  in the `task_order` list. A swap is only valid if it does not violate any precedence constraints  $Q$ , such that no predecessor task is shifted after its successor.
2. **Neighborhood  $N_2$  (Single Task Team Reassignment):**  $N_2(s)$  consists of all states where a single task  $t_i$  is reassigned to an alternative valid team  $j' \neq a_i$  from the qualified team set. This structure targets assignment costs without changing the task order sequence.
3. **Neighborhood  $N_3$  (Composite Sequence Swap and Reassignment):**  $N_3(s)$  represents a higher-order neighborhood designed for broader exploration. A state  $s' \in N_3(s)$  is obtained by swapping two non-adjacent tasks in the task sequence (with precedence validation) and reassigning one of those swapped tasks to a different qualified team.



## 6.2.5 Shaking Procedure

The Shaking phase acts as the diversification mechanism in VNS, helping the search escape from low-quality local minima. When the search stalls in a local minimum across smaller neighborhoods, a state  $s'$  is selected at random from the current neighborhood  $N_k(s)$ . This random variation shifts the candidate solution to a new region in the search space, with the search radius proportional to the neighborhood index  $k$ .

### Pseudo Code

```

1  Helper Function: Precedence Feasibility Check
2  1: function IS_PRECEDENCE_FEASIBLE(task_order, precedence)
3  2:     pos <- {task: idx for idx, task in enumerate(task_order)}
4  3:     for (i, j) in precedence do
5  4:         if pos[i] >= pos[j] then
6  5:             return False
7  6:         end if
8  7:     end for
9  8:     return True
10 9: end function

1  Operation: Shaking Step
2  1: function SHAKE(state, k, precedence, available_team)
3  2:     new_state <- copy(state)
4  3:     if k == 1 then
5  4:         idx <- random_integer(0, length(state["task_order"]) - 2)
6  5:         swap new_state["task_order"][idx], new_state["task_order"]
7  6:         if not IS_PRECEDENCE_FEASIBLE(new_state["task_order"],
8  7:             precedence) then
9  8:             revert swap
10 9:         end if
11 10:     else if k == 2 then
12 11:         idx <- random_integer(0, length(state["task_order"]) - 1)
13 12:         task <- state["task_order"][idx]
14 13:         if length(available_team[task]) > 1 then
15 14:             current_team <- state["team_assignment"][idx]
16 15:             new_state["team_assignment"][idx] <- random_choice(
17 16:                 available_team[task] except current_team)
18 17:         end if
19 18:     else if k == 3 then
20 19:         idx1, idx2 <- select two distinct random indices
21 20:         swap new_state["task_order"][idx1], new_state["task_order"]
22 21:         if IS_PRECEDENCE_FEASIBLE(new_state["task_order"],
23 22:             precedence) then
24 23:             task <- new_state["task_order"][idx1]
25 24:             if length(available_team[task]) > 1 then

```

```

23:         current_team <- state["team_assignment"][idx1]
24:         new_state["team_assignment"][idx1] <-
    random_choice(available_team[task] except current_team)
25:     end if
26:     else
27:         revert swap
28:     end if
29: end if
30: return new_state
31: end function

```

### 6.2.6 Local Search via Variable Neighborhood Descent

Following the shaking step, a local search phase is executed starting from the shaken solution  $s'$  to find a local optimum  $s''$ . While basic local searches utilize a single neighborhood, this implementation integrates Variable Neighborhood Descent (VND). VND systematically scan the neighborhood structures. VND iterates through the neighborhood structures sequentially, descending toward a local minimum across all defined spaces.

#### Pseudo Code

```

1 Operation: Variable Neighborhood Descent (VND) Local Search
2 1: function VND_LOCAL_SEARCH(state, precedence, available_team, cost
    , duration)
3 2:     current_state <- state
4 3:     best_fit <- CALC_FITNESS(current_state, cost, duration)
5 4:     l <- 1
6 5:     l_max <- 2
7 6:     while l <= l_max do
8 7:         improved <- false
9 8:         if l == 1 then
10 9:             for idx in range(0, length(current_state["task_order
    "]) - 1) do
11 10:                 neighbor <- copy(current_state)
12 11:                 swap neighbor["task_order"][idx], neighbor["
    task_order"][idx + 1]
13 12:                 if IS_PRECEDENCE_FEASIBLE(neighbor["task_order"],
    precedence) then
14 13:                     fit <- CALC_FITNESS(neighbor, cost, duration)
15 14:                     if fit < best_fit then
16 15:                         current_state <- neighbor
17 16:                         best_fit <- fit
18 17:                         improved <- true
19 18:                     end if
20 19:                 end if
21 20:             end for
22 21:         else if l == 2 then

```

```

23 22:          for idx in range(0, length(current_state["task_order
24      "])) do
25 23:          task <- current_state["task_order"][idx]
26 24:          current_team <- current_state["team_assignment"][
27      idx]
28 25:          for team in available_team[task] do
29 26:              if team != current_team then
30 27:                  neighbor <- copy(current_state)
31 28:                  neighbor["team_assignment"][idx] <- team
32 29:                  fit <- CALC_FITNESS(neighbor, cost,
33      duration)
34 30:                  if fit < best_fit then
35 31:                      current_state <- neighbor
36 32:                      best_fit <- fit
37 33:                      improved <- true
38 34:                  end if
39 35:              end if
40 36:          end for
41 37:      end for
42 38:  end if
43 39:  if improved then
44 40:      l <- 1
45 41:  else
46 42:      l <- l + 1
47 43:  end if
48 44:  end while
49 45:  return current_state
50 46: end function

```

### 6.2.7 Neighborhood Change and Acceptance Criterion

VNS coordinates the transition between exploration and exploitation using a deterministic neighborhood change strategy. Let  $f(s)$  denote the lexicographical fitness function, returning the objective tuple:

$$f(s) = (M - \text{scheduled\_tasks}, \text{completion\_time}, \text{total\_cost})$$

The fitness of the local optimum  $f(s'')$  is compared with the current best solution  $f(s^*)$ . If  $f(s'') < f(s^*)$ , which means that the local search discovered a better schedule, the current is updated ( $s^* \leftarrow s''$ ) and the neighborhood index is reset to the smallest structure ( $k \leftarrow 1$ ) to perform a detailed improvement. If no improvement is achieved, the neighborhood index increases ( $k \leftarrow k + 1$ ), which broadens the shaking radius for the next iteration.

## 6.2.8 Main Loop

### Pseudo Code

```

1 Operation: Main VNS Loop
2 1: function MAIN_VNS
3 2:   start_time <- time.time()
4 3:   max_time <- 1.0
5 4:   max_iter <- 10000
6 5:   k_max <- 3
7 6:   curr <- GREEDY_CONSTRUCT(num_task, precedence, graph,
   team_available_time, available_team, task_duration, cost)
8 7:   best <- curr
9 8:   f_best <- CALC_FITNESS(best, cost, duration)
10 9:   iter <- 0
11 10:  while iter < max_iter do
12 11:    if time.time() - start_time > max_time then
13 12:      break
14 13:    end if
15 14:    k <- 1
16 15:    while k <= k_max do
17 16:      candidate <- SHAKE(curr, k, precedence,
   available_team)
18 17:      candidate_opt <- VND_LOCAL_SEARCH(candidate,
   precedence, available_team, cost, duration)
19 18:      f_cand <- CALC_FITNESS(candidate_opt, cost, duration)
20 19:      if f_cand < f_best then
21 20:        curr <- candidate_opt
22 21:        best <- candidate_opt
23 22:        f_best <- f_cand
24 23:        k <- 1
25 24:      else
26 25:        k <- k + 1
27 26:      end if
28 27:    end while
29 28:    iter <- iter + 1
30 29:  end while
31 30:  return best
32 31: end function

```

## 6.3 Results

This section compares the judge scores obtained by our two implemented metaheuristic algorithms: Iterated Local Search (ILS) and Variable Neighborhood Search (VNS), against the reference solutions.

Each result is reported as:

(tasks scheduled, completion time, total cost)

This corresponds to the fitness criteria defined in the state decoding and fitness calculation sections. For readability, the table shows the number of scheduled tasks directly, while the internal fitness minimizes the number of unscheduled tasks.

**Comparison rule.** All algorithms are compared lexicographically:

more scheduled tasks  $\rightarrow$  smaller completion time  $\rightarrow$  lower total cost.

In this updated dataset, both metaheuristics successfully match the exact number of scheduled tasks achieved by the Jury across all ten instances. Consequently, the completion time and total operational cost serve as the primary differentiators to determine algorithmic superiority.

### 6.3.1 Score Comparison

| No. | Tasks | Teams | Prec. | Jury              | ILS               | VNS               |
|-----|-------|-------|-------|-------------------|-------------------|-------------------|
| 1   | 5     | 6     | 4     | (5, 235, 145)     | (5, 235, 145)     | (5, 235, 145)     |
| 2   | 10    | 15    | 6     | (10, 210, 430)    | (10, 190, 480)    | (10, 190, 480)    |
| 3   | 100   | 80    | 100   | (100, 550, 4510)  | (100, 550, 1900)  | (100, 550, 1900)  |
| 4   | 200   | 80    | 150   | (200, 370, 13240) | (200, 370, 10760) | (200, 370, 6250)  |
| 5   | 500   | 200   | 650   | (493, 840, 38150) | (493, 840, 37360) | (493, 780, 25430) |
| 6   | 600   | 300   | 750   | (599, 550, 42300) | (599, 550, 41850) | (599, 550, 24020) |
| 7   | 700   | 300   | 750   | (657, 510, 46270) | (657, 510, 45840) | (657, 500, 24930) |
| 8   | 800   | 400   | 750   | (786, 530, 63960) | (786, 530, 63350) | (786, 520, 40500) |
| 9   | 900   | 400   | 750   | (884, 820, 74300) | (884, 820, 74050) | (884, 810, 44030) |
| 10  | 1000  | 400   | 1750  | (980, 770, 68520) | (980, 770, 68370) | (980, 770, 50000) |

### 6.3.2 Observations

- **All methods match the jury in scheduled-task count:** This means the search mainly affects completion time and total cost, rather than the number of feasible tasks.
- **Significant Cost Optimization with VNS:** While both metaheuristics manage to lower total costs compared to the Jury starting from Instance 3, VNS demonstrates a

massive advantage over ILS. For instance, on the largest test case (Instance 10), VNS reduces the cost to 50,000 compared to ILS's 68,370 and the Jury's 68,520.

- **Improvements in Completion Time:** ILS generally matches the Jury's completion time, showing minor improvements only on Instance 2. Additionally, on Instances 5, 7, 8, and 9, VNS manages to finish earlier than both ILS and the Jury.
- **Algorithmic Efficiency:** The multi-neighborhood exploration of VNS is highly effective at escaping local optima, so that it outperforms ILS in refining the initial schedules, making VNS the lexicographical winner in this evaluation.

# Chapter 7

## Comparision

### 7.1 Solution Quality and Optimality

- **Exact Methods:** Mathematically find the global optimal solution within the bounded feasible region.
- **Heuristics:** Prioritize speed over precision, offering “good enough” solutions based on intuitive rules but providing no performance bounds or optimality guarantees, which often leaves them trapped in local optima.
- **Meta-heuristics:** Do not guarantee strict global optimality; however, they employ sophisticated exploration and exploitation mechanisms to escape local optima, consistently yielding high-quality, near-optimal solutions for complex search spaces.

### 7.2 Execution Time and Computational Complexity

- **Exact Methods:** The execution time scales exponentially or factorially, and it will increase intensively when the input is larger; therefore, they are impractical for real-time decision-making.
- **Heuristics:** Computationally trivial, generally executing within milliseconds as they follow deterministic, one-pass rule sets (e.g. greedy approaches).
- **Meta-heuristics:** Their execution times are bounded by user-defined termination criteria, such as maximum generations or stagnation limits, effectively bridging the gap between exact methods and simple heuristics.

## 7.3 Scalability and Generality

- **Exact Methods:** Suffer from poor scalability and generality, and are strictly confined to small or moderate problem instances.
- **Heuristics:** Exhibit excellent scalability, remaining robust and capable of handling large-scale problems where exact solvers fail to converge; however, each problem has a particular heuristic method to access, make it hard to generalize.
- **Meta-heuristics:** Highly scalable and robust when applied to ultra-large-scale problems; moreover, it is very easy to generalize since Meta-heuristics just concern on the quality of solutions but not deeply explore the concept and structure of problem.

## 7.4 Implementation and Tuning Effort

- **Exact Methods:** The main work lies in writing down the correct mathematical equations for the model. Once the model is ready, existing software solvers (like ORTools) can be applied to do the heavy lifting and find the solution.
- **Heuristics:** The easiest to program. They require very little development time, simple logic design, and very low computer power to run.
- **Meta-heuristics:** Require a fair amount of coding to set up the basic problem structure. However, the most time-consuming part is testing and adjusting different settings through trial and error to get the best results.